

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»
(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**«АВТОМАТИЗИРОВАННАЯ СИСТЕМА ПО ОТСЛЕЖИВАНИЮ
ВЫПОЛНЕНИЯ ЗАДАЧ В КОМПАНИИ»**

по направлению подготовки
09.03.01 Информатика и вычислительная техника
(уровень бакалавриата)
профиль «Информационные системы»

Студент _____ М.П. Федулов
(подпись, дата)

Руководитель ВКР,
К.Т.Н. _____ Я.В. Соловьева
(подпись, дата)

Нормоконтролер _____ А.А. Тарасов
(подпись, дата)

Самара 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»
(Самарский университет)

Институт информатики и кибернетики

Кафедра информационных систем и технологий

УТВЕРЖДАЮ

Зав. кафедрой ИСТ

_____ И.А. Лёзин

« ____ » _____ 20__ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
БАКАЛАВРА

студенту Федулову Михаилу Павловичу

группа № 6404-090301D

Тема работы: «Автоматизированная система по отслеживанию выполнения задач в компании»

утверждена приказом по университету № 118-Т от «26» февраля 2026 г.

Исходные данные к ВКР:

- 1) характеристика объекта автоматизации: система по отслеживанию выполнения задач и управление ими;
- 2) требования к программному обеспечению: операционная система Windows, языки программирования Java и TypeScript;
- 3) функции, реализуемые системой: просмотр, удаление, изменение задач, а также работа с цепочками задач, аутентификация пользователей и

создание новых, разграничение прав пользователей в зависимости от их роли.

Перечень вопросов, подлежащих разработке в ВКР:

- 1) анализ предметной области и существующих систем-аналогов;
- 2) разработка логического проекта системы;
- 3) разработка логической и физической моделей базы данных;
- 4) разработка и реализация программного обеспечения;
- 5) описание контрольного примера.

Руководитель работы

доцент кафедры ИСТ _____ Я.В. Соловьева

(подпись)

« ____ » _____ 20 ____ г.

Задание принял к исполнению _____

(подпись)

М.П. Федулов

« ____ » _____ 20 ____ г.

РЕФЕРАТ

Пояснительная записка: 104 страницы, 50 рисунков, 24 источника, 2 приложения.

Графическая часть: 18 слайдов Microsoft PowerPoint.

СИСТЕМА ПО УПРАВЛЕНИЮ ЗАДАЧАМИ, МИКРОФРОНТЕНДЫ,
POSTGRESQL, SPRING, ANGULAR, REACT, MODULE FEDERATION

В выпускной квалификационной работе разработана автоматизированная система по отслеживанию задач «Менеджер задач» с использованием микрофронтендной архитектуры.

Цель работы – автоматизация и упрощение процесса контроля и ведения задач.

В рамках работы был проведен анализ предметной области и существующих систем-аналогов, разработан проект системы по методологии UML при помощи инструмента draw.io, так и модель базы данных системы с помощью ER-диаграмм в среде ERwin Data Modeler 7. Для хранения данных используется база данных PostgreSQL, как одно из надежных решений. Программная реализация серверной части выполнена на языке программирования Java с использованием фреймворка Spring, клиентской части – на языке TypeScript с использованием фреймворка Angular и библиотеки React. Используемые среды разработки – OpenIDE 2025.2.4 и Visual Studio Code.

СОДЕРЖАНИЕ

Введение.....	7
1 Постановка задачи и обоснование методов решения задачи	8
1.1 Описание предметной области и постановка задачи	8
1.2 Анализ существующих методов решения задачи и обоснование выбора реализуемого метода.....	8
1.3 Обзор имеющихся систем-аналогов	9
2 Проектирование и программная реализация метода решения задачи.....	13
2.1 Описание методологии проектирования системы.....	13
2.2 Выбор комплекса программных средств.....	13
2.2.1 Выбор среды проектирования	13
2.2.2 Выбор языка программирования.....	15
2.2.3 Выбор среды разработки.....	16
2.2.4 Выбор операционной системы	17
2.2.5 Выбор программных технологий	17
2.2.6 Выбор системы управления базой данных	19
2.3 Архитектура системы	19
2.4 Проектирование информационно-логической системы	21
2.4.1 Диаграмма вариантов использования.....	21
2.4.2 Диаграмма классов системы.....	24
2.4.3 Диаграмма состояний	24
2.4.4 Диаграмма деятельности.....	25
2.4.5 Диаграмма последовательности	25
2.5 Структура базы данных	26
2.5.1 Логическая структура базы данных.....	26
2.5.2 Физическая структура базы данных	27
2.6 Описание основных алгоритмов системы.....	28
3 Вычислительные эксперименты проверки эффективной программной реализации метода решения задачи	35
Заключение	55

Список использованных источников	56
Приложение А Руководство пользователя	59
Приложение Б Листинг программы	68

ВВЕДЕНИЕ

В современных условиях разработки программного обеспечения и командной работы эффективность процессов напрямую зависит от качества организации задач. Студенты, работающие над групповыми проектами, и небольшие рабочие команды часто сталкиваются с проблемой хаотичного распределения обязанностей. Использование мессенджеров и электронных таблиц для трекинга прогресса приводит к потере информации, дублированию задач и срыву дедлайнов. В то же время существующие на рынке корпоративные системы управления проектами зачастую обладают избыточным функционалом, сложны в освоении или требуют значительных финансовых затрат для небольших групп.

Для решения проблемы децентрализованного управления задачами и повышения прозрачности рабочих процессов было принято решение разработать веб-приложение по управлению задачами. Данная система представляет собой легковесный инструмент для планирования и контроля выполнения задач, адаптированный под малых команд.

Приложение предоставляет пользователям возможность создавать задачи, отслеживать их статус в реальном времени и взаимодействовать в едином информационном пространстве. Важной особенностью системы является строгое разграничение прав доступа: наличие ролевой модели позволяет гибко управлять командой, назначать ответственных и контролировать выполнение критически важных задач, обеспечивая при этом безопасность данных.

1 Постановка задачи и обоснование методов решения задачи

1.1 Описание предметной области и постановка задачи

Автоматизированная информационная система по отслеживанию задач предназначена для упрощения процессов отслеживания и управления.

Система должна предоставлять возможность регистрации пользователя и его авторизацию в приложении. Система должна иметь функции по работе с задачами, такими как: создание новых задач, редактирование или удаление существующих. Еще система должна иметь функции по работе с цепочками задач.

Также в системе должен быть реализован ролевой доступ, позволяющий выполнять определенные функции определенным ролям. Пользователь может входить в свой профиль, создавать и изменять свои задачи. Модератор, как и обычный пользователь, может работать со своими задачами и дополнительно взаимодействовать с задачами других пользователей, включая удаление. Администратор имеет максимальный уровень доступа ко всем методам работы с задачами, как и остальные роли, но дополнительно может изменять пароли от профилей пользователей и удалять их.

Автоматизированная информационная система должна быть эффективной и отказоустойчивой.

1.2 Анализ существующих методов решения задачи и обоснование выбора реализуемого метода

Традиционно планирование рабочих задач и отслеживание прогресса в небольших командах или студенческих проектах ведется вручную: через мессенджеры, электронные таблицы или разрозненные заметки. Такой подход работает на начальных этапах, но быстро приводит к потере информации, дублированию поручений и срыву сроков, особенно когда над проектом одновременно работает несколько исполнителей.

Крупные организации решают эту проблему внедрением специализированных корпоративных систем управления проектами. Подобные платформы предоставляют детальный контроль, аналитику и

интеграции, однако требуют значительных финансовых вложений, времени на обучение сотрудников и выделения отдельных администраторов для поддержки инфраструктуры.

Небольшие рабочие группы, стартапы и учебные коллективы не могут позволить себе дорогостоящие лицензии или сложную настройку корпоративных инструментов, несмотря на острую потребность в порядке и прозрачности рабочих процессов. Таким командам необходимы легкие, доступные решения, которые не перегружают интерфейс избыточным функционалом и запускаются без длительной подготовки.

Альтернативой становится веб-приложение для управления задачами, развертываемое на локальном сервере или в облачной среде. Подобный инструмент должен покрывать базовые потребности командной работы: создание и назначение задач, отслеживание статусов выполнения, разграничение прав доступа между участниками и централизованное хранение истории изменений.

Разработка специализированного веб-приложения предпочтительнее использования универсальных конструкторов или таблиц по нескольким причинам:

- единое информационное пространство, доступное всем участникам команды с любого устройства;
- отсутствие ежемесячных платежей за лицензии и сокращение времени на согласование действий;
- автоматическая фильтрация и сортировка задач по статусу, дате создания или ответственному лицу;
- гибкая ролевая модель, позволяющая администраторам управлять учетными записями и статусами, а модераторам – контролировать общий поток работ без риска случайного изменения критических данных.

1.3 Обзор имеющихся систем-аналогов

В ходе работы были проанализированы системы-аналоги, предоставляющие компаниям возможности по управлению задачами и

организации рабочего процесса.

Trello – гибкий инструмент для управления проектами. Команды используют его, чтобы генерировать идеи, планировать совместную работу, выстраивать процессы и отслеживать прогресс. Интерфейс наглядный и удобный: ничего лишнего, только то, что помогает двигаться вперёд. Сервис одинаково хорошо закрывает и масштабные проекты, и повседневные задачи – от первого мозгового штурма до финальной реализации.

Работает Trello по принципу канбан-досок: задачи – это карточки, которые перетаскиваешь между колонками по мере выполнения. Интерфейс достаточно простой и не требует долгого обучения. Система подойдёт для личных списков дел или небольших команд, которые ценят простоту запуска.

В бесплатной версии ограничено количество рабочих пространств. Настройки прав доступа – только базовые: администратор или участник. Гибкая фильтрация и сортировка доступны лишь в платных тарифах. Данные хранятся на облачных серверах поставщика, поэтому локальное развёртывание невозможно без миграции. [1] Интерфейс сервиса представлен на рисунке 1.

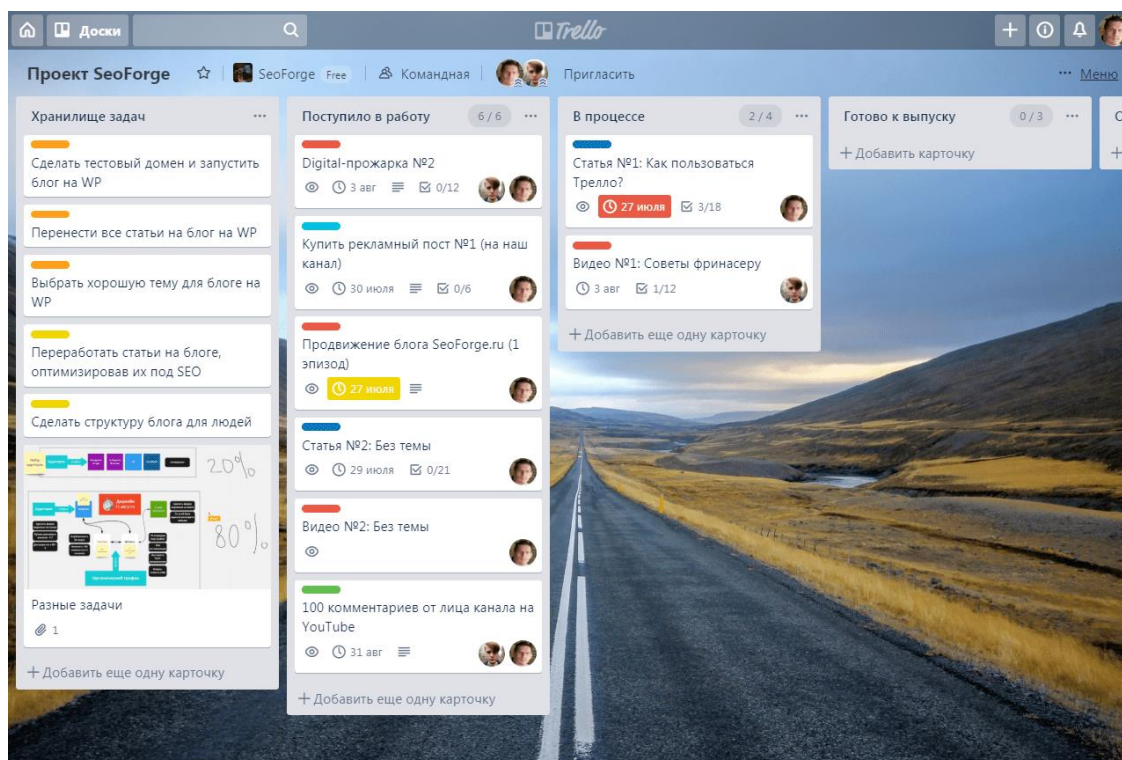


Рисунок 1 – Интерфейс сервиса Trello

Jira – это инструмент управления проектами, который помогает оптимизировать работу команды. Сервис работает как диспетчер задач: показывает, какие проекты запущены и сколько людей на них занято. Проджект-менеджер использует Jira, чтобы распределять задачи и планировать загрузку сотрудников.

Бесплатная версия рассчитана на команду до 10 человек – поэтому Jira так любят стартапы. Для более крупных команд есть тариф Standard: \$7 за пользователя, 250 ГБ хранилища и техподдержка в рабочие часы. В Premium за \$14 с пользователя вы получаете безлимитное хранилище и круглосуточную поддержку. Оба тарифа позволяют подключить до 20 000 человек, оплата – ежемесячно или раз в год.

Самый дорогой вариант – Enterprise. Его оплачивают только ежегодно и выбирают, когда нужны особые требования к безопасности или масштабированию. [2] Интерфейс Jira представлен на рисунке 2.

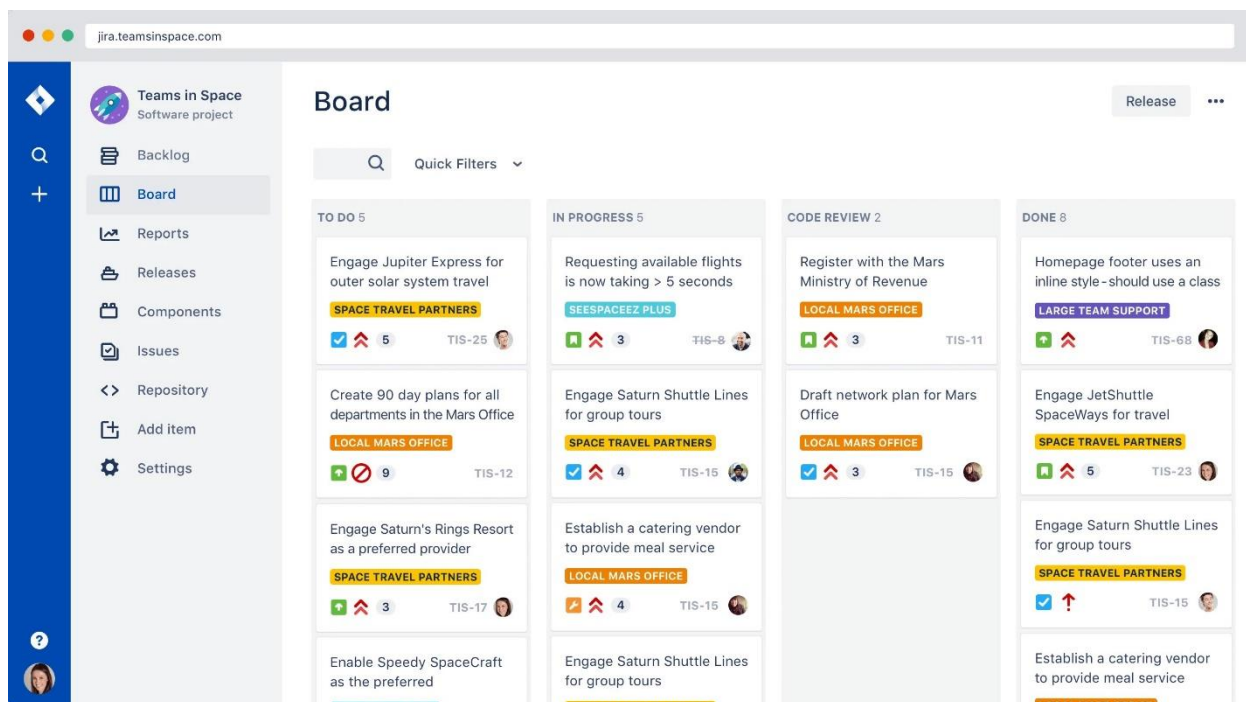


Рисунок 2 – Интерфейс Jira

Asana – веб-приложение для управления проектами и задачами. Оно помогает командам планировать, отслеживать и организовывать процессы: каждый участник видит, кто за что отвечает, какие задачи сейчас в работе и на

каком этапе находится проект. Сервис позволяет контролировать сроки, автоматизировать рутинные задачи и наладить совместную работу. [3] Интерфейс веб-приложения представлен на рисунке 3.

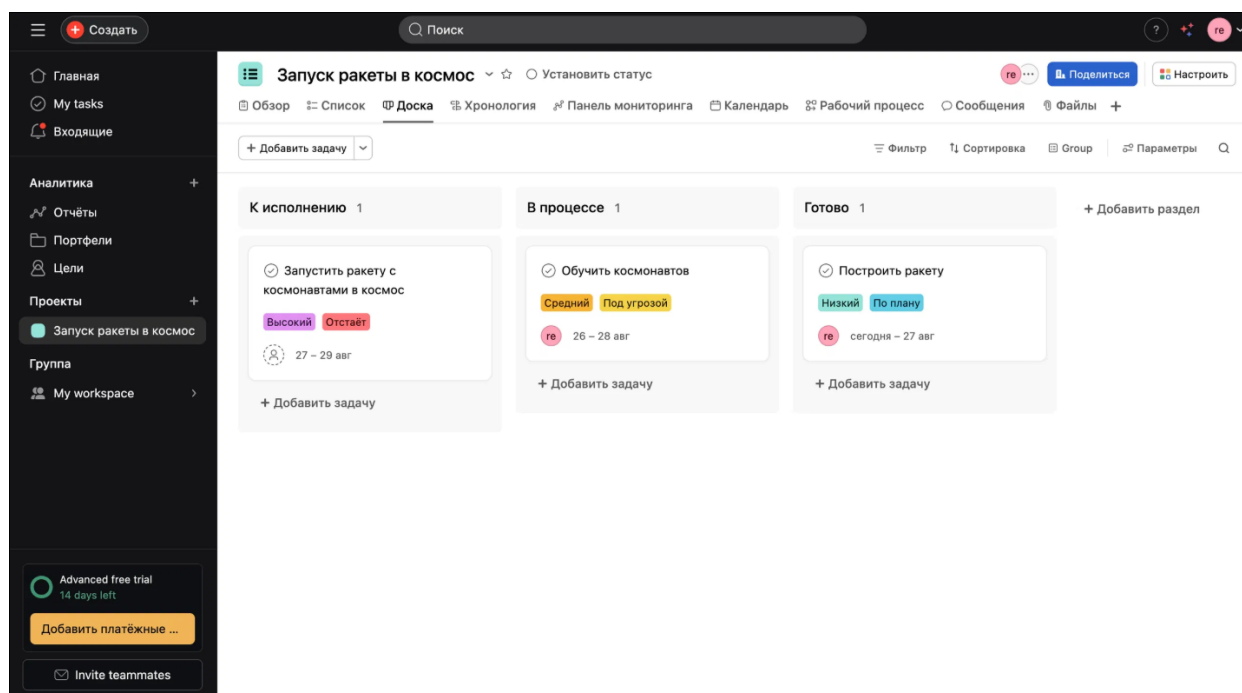


Рисунок 3 – Интерфейс веб-приложения Asana

Опираясь на результаты анализа систем-аналогов, можно сделать вывод, что ни одна из перечисленных систем не обеспечивает полный набор функций, необходимый для компаний. Многие решения требуют наличия определенной подписки, бесплатные версии могут не удовлетворять потребностям команды, или решения могут не работать при отсутствии интернета.

Следовательно, было принято решение о разработке собственной системы по управлению задачами. Данное решение будет экономически выгодным и адаптивным под конкретные нужды компании.

2 Проектирование и программная реализация метода решения задачи

2.1 Описание методологии проектирования системы

UML (Unified Modeling Language) – унифицированный язык моделирования. Это графический язык: каждой фигуре, символу, стрелке или их комбинации здесь присвоено конкретное значение. Благодаря этому схема на UML становится понятной любому, кто знаком с правилами языка.

По сути, UML – это свод правил для рисования схем. Зная их, можно быстро создавать универсальные графические представления сложных структур и процессов. Поэтому IT-специалисты так часто обращаются к UML при разработке ПО: язык помогает проектировать и описывать процессы наглядно.

UML нужен, чтобы описывать и визуализировать абстрактные модели. На практике это может быть модель объекта или модель бизнес-процесса. Диаграмму рисуют либо по готовому объекту (для анализа), либо на этапе проектирования (чтобы отладить процесс до запуска). Такие схемы используют в проектировании, презентациях, технической документации.

UML обладает преимуществами:

- стандартизация: диаграмму поймёт любой специалист, владеющий языком, независимо от команды или компании;
- полнота: в языке уже есть обозначения для всех типов сущностей, которые обычно нужны в схемах;
- распространенность: UML используют в IT, менеджменте, инженерии и других областях – язык знаком специалистам по всему миру;
- инструменты автоматизации: некоторые среды (например, Microsoft Visual Studio 2010) умеют генерировать UML-диаграммы напрямую из кода.

[4]

2.2 Выбор комплекса программных средств

2.2.1 Выбор среды проектирования

Информационные технологии развиваются быстро, экономические системы усложняются – автоматизация разработки и сопровождения ПО

перестала быть опцией, стала необходимостью. Здесь на помощь приходят CASE-средства.

CASE (Computer-Aided Software Engineering) – это набор программных инструментов для автоматизации жизненного цикла информационных систем. Они поддерживают команду на всех этапах: от сбора требований и проектирования до кодирования, тестирования и внедрения.

Инструменты решают разные задачи: моделируют бизнес-процессы, проектируют базы данных, генерируют код, помогают с документированием. [5]

При проектировании информационной системы было выбрано приложение Erwin Data Modeler.

AllFusion ERwin Data Modeler помогает проектировать, документировать и поддерживать базы данных, хранилища и витрины данных через визуальное моделирование. Инструментом пользуются разработчики, аналитики, проектировщики баз данных и руководители проектов.

Приложение поддерживает прямое и обратное проектирование: можно создать базу данных из готовой модели или, наоборот, сгенерировать модель по существующей базе. Работает с 20 типами СУБД. Реализует методологию структурного моделирования SADT и нотации IDEF1x, IE, Dimensional. Позволяет подробно документировать структуру базы.

Ещё одна сильная сторона – возможность создавать шаблоны триггеров и хранимых процедур. На их основе инструмент генерирует серверный код под разные СУБД. Таких функций нет у большинства аналогов, что делает ERwin удобным выбором для сложных проектов. [6]

Для построения диаграмм UML был использован сервис Draw.io.

Draw.io – это бесплатный онлайн-сервис, который позволяет создавать и редактировать диаграммы, различные схемы и другие визуальные представления данных. Благодаря его простоте, можно быстро создать диаграмму любого типа и наглядно представить информацию. С его помощью можно описать любой бизнес-процесс и построить архитектуру приложений.

2.2.2 Выбор языка программирования

При разработке информационной системы были выбраны языки Java (для серверной части) и TypeScript (для клиентской части).

Java – это объектно-ориентированный язык программирования, разработанный компанией Sun Microsystems и впервые выпущенный в 1995 году. Основные принципы Java включают кроссплатформенность, надёжность, безопасность и простоту. Ключевая особенность языка – «пиши один раз, запускай где угодно» (Write Once, Run Anywhere), благодаря использованию виртуальной машины Java (JVM), которая интерпретирует байт-код, независимо от операционной системы. Java поддерживает автоматическую сборку мусора, исключения, многопоточность и богатую стандартную библиотеку. Java используют в разработке корпоративных приложений, Android-приложений, веб-сервисов, а также в научных и образовательных целях. С появлением более современных языков, Java остаётся одним из самых популярных и востребованных языков программирования в мире. [7]

TypeScript – надстройка над JavaScript. Он помогает писать безопасный и аккуратный код: ошибки появляются ещё на этапе разработки, до запуска программы. Синтаксис почти не отличается от JS, а готовый код компилируется в обычный JavaScript. На этом языке собирают любые продукты – от простых сайтов до сложных приложений.

Разработчики берут TypeScript, чтобы обойти ограничения JavaScript. В оригинале нет строгой типизации, а отладка и поддержка больших проектов требуют больше усилий. TypeScript закрывает эти пробелы: добавляет проверку типов, упрощает чтение кода и делает архитектуру управляемой. В результате продукт получается стабильнее, а разработка – прозрачнее.

Основные преимущества языка:

- статическая типизация, в отличие от JavaScript;
- компиляция кода до его выполнения;
- совместимость с JavaScript;

- полноценное объектно-ориентированное программирование (ООП);
- обширная документация.

На написание кода уходит больше времени из-за необходимости описывать типы. Иногда возникают конфликты с устаревшими или самописными библиотеками JavaScript, которые не адаптированы под строгую типизацию. [8]

2.2.3 Выбор среды разработки

В качестве среды разработки были выбраны OpenIDE 2025.2.4 (для серверной части) и Visual Studio Code (для клиентской части).

OpenIDE – бесплатная IDE с открытым исходным кодом, построенная на базе IntelliJ IDEA Platform.

Вся инфраструктура для сборки и работы среды размещена в России. Когда OpenIDE отправляет статистику, ищет обновления или подключает плагины, она обращается только к серверам на территории РФ.

В маркетплейсе с первого дня доступно более 300 плагинов – можно сразу расширять функционал под свои задачи. [9]

Visual Studio Code (VS Code) от Microsoft – кроссплатформенный редактор кода. Он работает на Windows, macOS и Linux. Внутри встроен инструментарий для написания, отладки и тестирования, а маркетплейс расширений позволяет собрать среду под конкретную задачу.

Редактор закрывает проекты разной сложности: от правки конфигурационных файлов до сборки веб- и мобильных приложений. Из коробки идут профили для JavaScript, TypeScript, HTML, CSS, Python, R, Java, Kotlin, Go, Rust, C/C++ и SQL. Если стек отличается, поддержку подключают через расширения.

Инструмент используют не только программисты. Веб-разработчики пишут в нём API и клиентскую часть продуктов. Системные администраторы автоматизируют инфраструктуру скриптами. Аналитики данных и специалисты по машинному обучению загружают датасеты, обрабатывают массивы и тренируют модели прямо в редакторе. [10]

2.2.4 Выбор операционной системы

В качестве операционной системы была выбрана Microsoft Windows 10.

Windows 10 – это операционная система для персональных компьютеров и рабочих станций, разработанная корпорацией Microsoft в рамках семейства Windows NT.

Операционная система Windows является самой распространенной как в России, так и в мире. Она предоставляет пользователям все необходимые средства для создания пользовательского интерфейса и поддерживает все популярные среды разработки программного обеспечения. [11]

2.2.5 Выбор программных технологий

Так как разработка представляет собой веб-приложение, были использованы следующие программные технологии: Spring Framework, Angular Framework, React и Module Federation.

Spring Framework – фреймворк для разработки Java-приложений, который предоставляет инструменты для создания сложных систем, например многопользовательских корпоративных веб-приложений. Фреймворк ускоряет создание сервисов, которые работают с базами данных и облачными платформами, состоят из независимых модулей и обмениваются информацией с пользователями по защищённым каналам. На чистой Java всё это тоже можно написать вручную, но Spring позволяет упростить и ускорить этот процесс. Готовые инструменты экономят время и позволяют сосредоточиться на уникальной логике продукта, а не на настройке инфраструктуры.

За эту особенность Spring иногда называют «фреймворком фреймворков». Он не просто предлагает собственную архитектуру, а включает проверенные сторонние решения: Struts, Hibernate, Tapestry, EJB, JSF и другие библиотеки.

На базе Spring пишут серверную часть для современных десктопных, мобильных и веб-клиентов. Инструмент масштабируется одинаково хорошо: подходит и для небольшого сервиса, и для крупной системы с десятками модулей и сотнями функций. Архитектура фреймворка упрощает

проектирование, сокращает объём шаблонного кода и берёт на себя связывание компонентов.

Популярные и часто используемые модули в Spring:

- Spring Web: для создания веб-приложений и RESTful-сервисов;
- Spring Data: для упрощения работы с реляционными и нереляционными базами данных, облачными сервисами и фреймворками map-reduce;
- Spring Security: комплексная система безопасности для Java-приложений, включающая аутентификацию, проверку прав доступа и поддержку современных механизмов безопасности (JWT, OAuth2);
- Spring Core: ядро фреймворка и основа, реализующая принципы инверсии контроля (Inversion of Control) и внедрения зависимостей (Dependency Injection). [12]

Для связи серверной части с базой данных использовался модуль Spring Data – Java Database Connectivity (JDBC).

Spring Data JDBC – это часть экосистемы Spring Data, которая предлагает простой подход к работе с реляционными базами данных с использованием обычного SQL и простых объектов Java. В отличие от Spring Data JPA, основанной на JPA (Java Persistence API) и фреймворках ORM (объектно-реляционного отображения), Spring Data JDBC ориентирована на более прямое и менее абстрактное взаимодействие с базой данных. [13]

Angular – фронтенд-фреймворк от Google, заточенный под сложные веб-приложения. Фреймворк написан на языке TypeScript и содержит в себе строгую модульную архитектуру. [14]

React.js – это библиотека для языка программирования JavaScript с открытым исходным кодом для разработки пользовательских интерфейсов. [15]

Module Federation – это подход, при котором можно разделить приложение на небольшие отдельные модули и в процессе работы объединять в единое приложение. Это одно из немногих рабочих решений для разработки микрофронтендов. [16]

2.2.6 Выбор системы управления базой данных

В качестве СУБД (системы управления базой данных) было выбрано PostgreSQL.

PostgreSQL – это объектно-реляционная система управления базами данных (ORDBMS), наиболее развитая из открытых СУБД в мире. Имеет открытый исходный код и является альтернативой коммерческим базам данных.

СУБД позволяет гибко управлять базами данных. С ее помощью можно создавать, модифицировать или удалять записи, отправлять транзакцию – набор из нескольких последовательных запросов на особом языке запросов SQL.

Сильная сторона PostgreSQL – поддержка множества типов данных. Помимо стандартных целых чисел, чисел с плавающей точкой, строк и булевых значений, система работает с денежными, геометрическими, перечисляемыми и бинарными типами. «Из коробки» доступны битовые строки, сетевые адреса, массивы (в том числе многомерные), композитные типы и другие сложные структуры. Есть встроенная поддержка XML, JSON и возможность использовать подходы, характерные для NoSQL-баз. [17]

2.3 Архитектура системы

Разрабатываемая система представляет собой веб-приложение с микрофронтендной архитектурой.

Веб-приложение – это клиент–серверное приложение, работающее через браузер по протоколам HTTP/HTTPS, с интерактивным интерфейсом и возможностью обработки данных на удаленном сервере. Оно состоит из фронтенда и серверной части с логикой и базами данных. [18]

Клиент-серверная архитектура представляет собой модель взаимодействия клиента и сервера. Клиент может взаимодействовать с приложением, чтобы получать и взаимодействовать с данными с сервера. Клиент формирует запрос, приложение принимает и обрабатывает его, при необходимости обращается к базе данных для получения данных.

Пример клиент-серверной архитектуры представлен на рисунке 4.

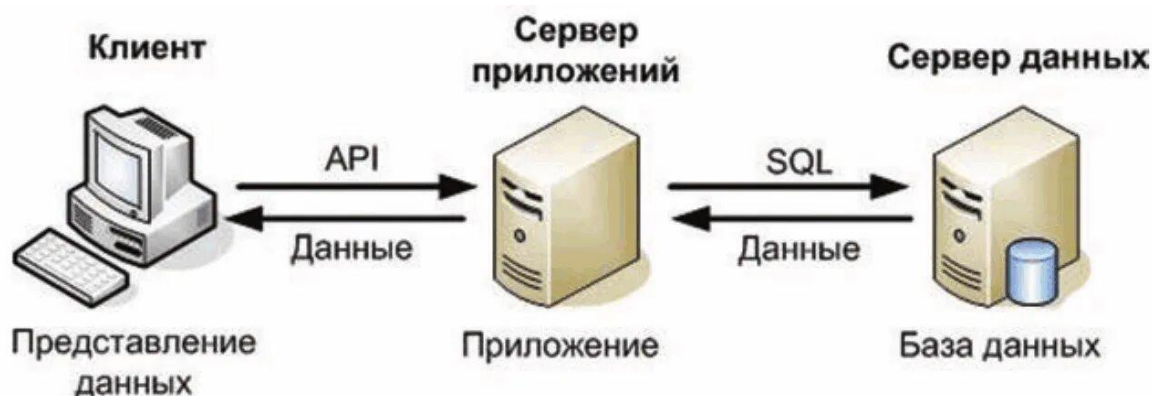


Рисунок 4 – Клиент-серверная архитектура

Фронтенд (англ. Frontend) – это разработка интерфейса и пользовательских функций. Сюда входит всё, что человек видит на экране и с чем взаимодействует: кнопки, формы, списки, меню.

Бэкенд (англ. Backend) – серверная логика, скрытая от пользователя. В отличие от фронтенда, этот код не загружается в браузер, а выполняется на сервере. Пользователь нажимает кнопку на фронтенде, а вся обработка данных и бизнес-логика происходят на стороне бэкенда. [19]

Монолитная архитектура – классический подход, когда весь код приложения упакован в один неразделимый пакет. Интерфейс, бизнес-логика и работа с данными живут в одном пространстве и функционируют как единое целое.

У такого решения есть очевидные плюсы. Проще писать и тестировать код, разворачивать приложение удобнее, а взаимодействие компонентов внутри процесса не тратит время на сетевые вызовы, что даёт высокую производительность. Однако с ростом проекта эти преимущества быстро тают. Монолит трудно масштабировать, сложно обновлять и поддерживать. Любое изменение затрагивает всю систему, а гибкость архитектуры снижается.

Для решения проблемы с объемом проекта можно применять микрофронтендный подход.

Микрофронтенд – это разделение монолитного фронта на отдельные кодовые базы, хранящиеся в отдельных репозиториях. По большей части этот

подход представляет собой организационное решение о том, как управлять сложностью разработки в большом проекте. Огромный «монолит» приложения можно разбить на несколько микрофронтендов, где каждая команда разработчиков будет отвечать за свою часть.

Стоит учесть, что приложение хоть и делится на несколько частей, важно не отходить от единого стиля и концепции всего «монолита». Примерная структура веб-приложения на микрофронтендах представлена на рисунке 5.

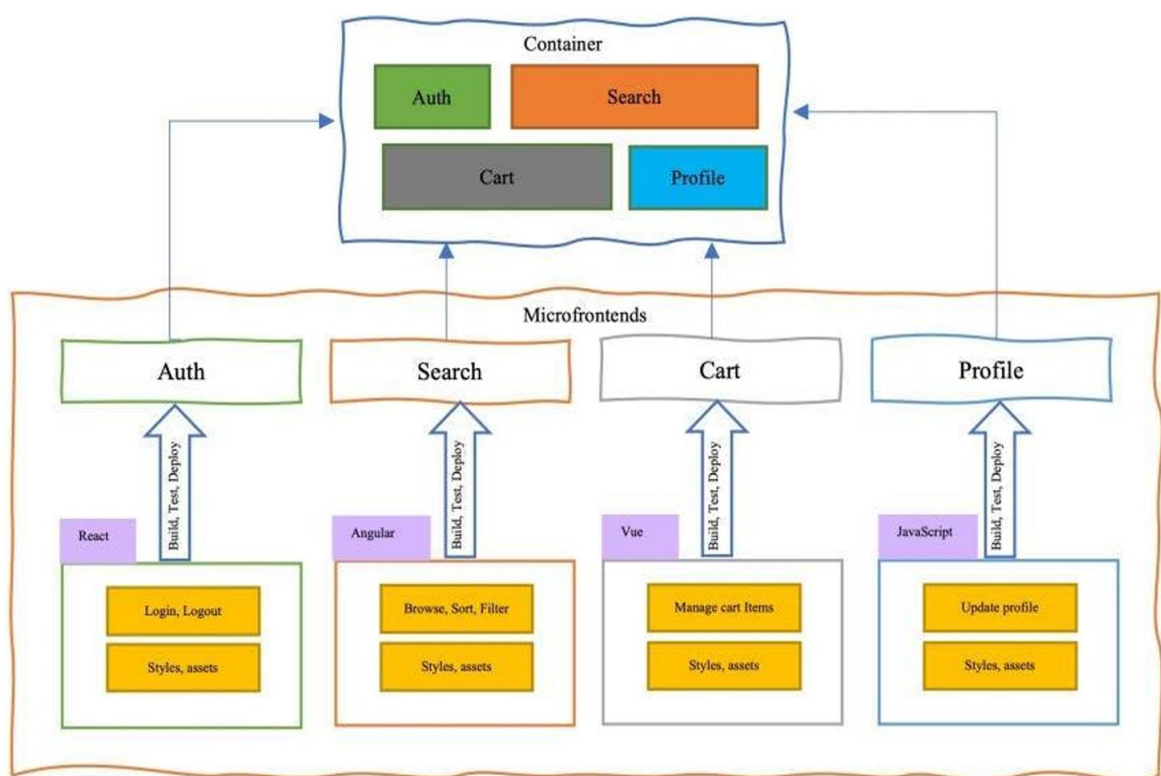


Рисунок 5 – Структура веб-приложения с микрофронтендной архитектурой

2.4 Проектирование информационно-логической системы

2.4.1 Диаграмма вариантов использования

Диаграмма вариантов использования (англ. use-case diagram) показывает, какие функции системы доступны разным группам пользователей. На ней видно, кто и как взаимодействует с продуктом: какие сценарии предусмотрены для каждого типа пользователей и какие задачи они могут решать. [20]

На рисунке 6 представлена диаграмма вариантов использования.



Рисунок 6 – Диаграмма вариантов использования

Для более детального описания взаимодействия пользователя используют сценарии использования - описание взаимодействия пользователя с системой для достижения конкретной цели.

Рассмотрим некоторые сценарии использования.

Сценарий для варианта использования: «Зарегистрироваться в системе»:

Вариант использования: Зарегистрироваться в системе.

Краткое описание: Позволяет пользователю зарегистрироваться в системе.

Актант: Пользователь.

Предусловие: Пользователь успешно попал на страницу авторизации в системе.

Основной поток событий:

- 1) пользователь наводится курсором по ссылке «Зарегистрироваться» в нижней части формы авторизации и нажимает на неё;
- 2) пользователь заполняет все необходимые поля и нажимает на кнопку «Зарегистрироваться»;

Альтернативы:

A1: пользователь видит перед собой ошибку, что пользователь с таким же логином уже существует;

A2: кнопка «Зарегистрироваться» неактивна, из-за нарушенной валидации.

- 3) в случае успеха, пользователь перенаправляется на страницу входа.

Постусловия: Пользователь успешно зарегистрировался и теперь может войти в систему.

Сценарий для варианта использования: «Удалить задачу другого пользователя»:

Вариант использования: Удалить задачу другого пользователя.

Краткое описание: Удаление задач может пригодиться, когда пользователь допустил ошибку или сделал задачу-дубликат другой.

Актант: Модератор.

Предусловие: Модератору сообщили о задаче-дубликате другого пользователя.

Основной поток событий:

- 1) модератор заходит в свой профиль;
- 2) перед модератором появляется список со всеми цепочками и задачами всех пользователей;
- 3) модератор выбирает группировку по пользователям;
- 4) модератор просматривает весь разгруппированный список задач выбранного пользователя;

Альтернативы:

A1: модератор не находит искомую задачу.

5) модератор нашел искомую задачу;

Альтернативы:

A1: модератор нажимает на кнопку изменения задачи и исправляет содержимое.

6) модератор нажимает на кнопку удаления задачи и подтверждает действие.

Постусловия: Искомая ошибочная задача найдена и удалена или изменена.

2.4.2 Диаграмма классов системы

Диаграмма классов отображает структуру системы, содержащей различные объекты и классы. Чаще всего используется, чтобы продемонстрировать иерархию классов внутри программы.

Диаграмма основных классов представлена на рисунке 7.

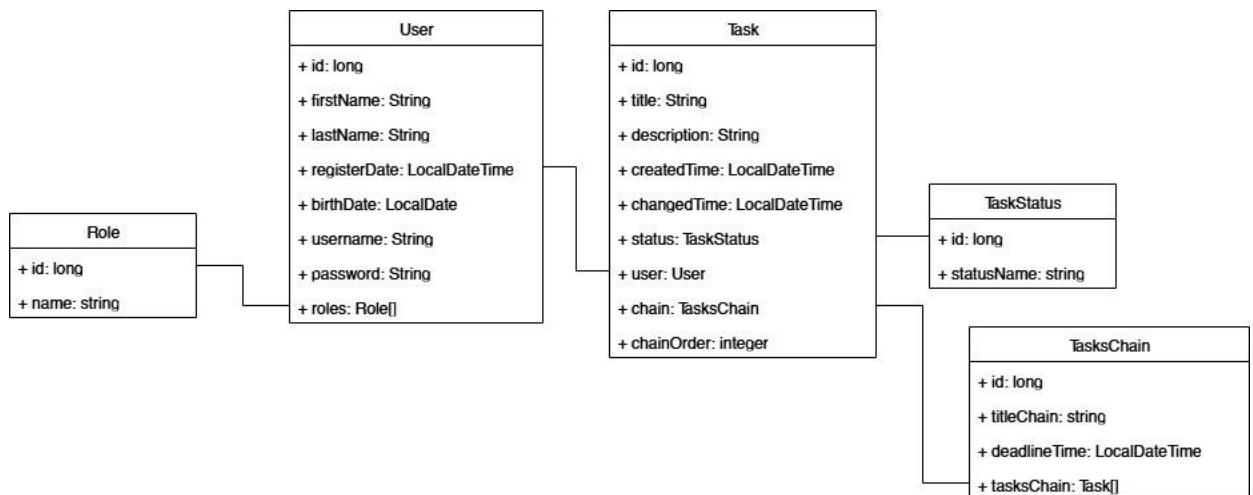


Рисунок 7 – Диаграмма классов

2.4.3 Диаграмма состояний

Диаграмма состояний (statechart diagram) – диаграмма, которая представляет конечный автомат, моделирующий процессы функционирования.

Главное назначение диаграммы – описать возможные последовательности состояний и переходов, которые в совокупности характеризуют поведение моделируемой системы в течение всего ее жизненного цикла.

Диаграмма состояний представляет динамическое поведение сущностей, на основе спецификации их реакции на восприятие некоторых конкретных событий. [21]

Диаграммы состояний для сценария «Получения списка задач» и сценария «Создание цепочки задач» представлены на рисунках 8 и 9.

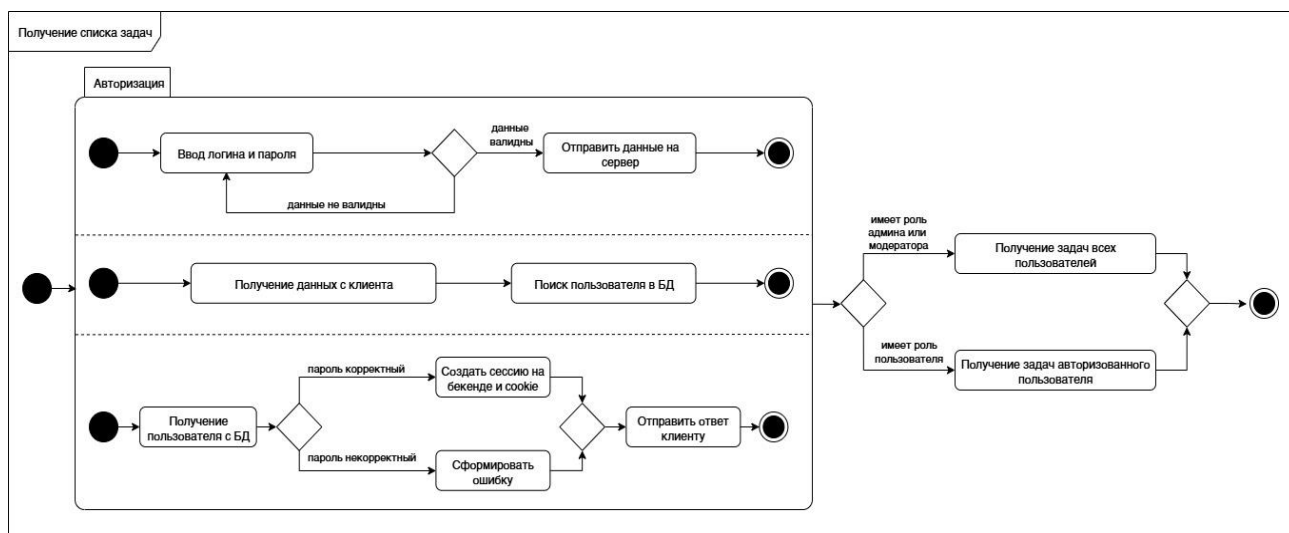


Рисунок 8 – Диаграмма состояний для сценария «Получение списка задач»

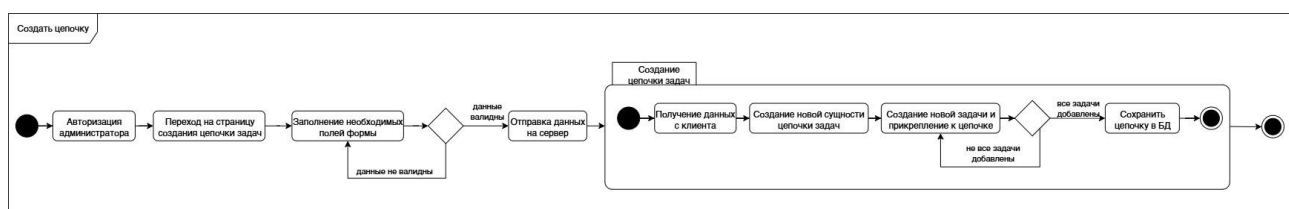


Рисунок 9 - Диаграмма состояний для сценария «Создать цепочку»

2.4.4 Диаграмма деятельности

Диаграммы деятельности UML показывают последовательность действий и поток управления в системе или процессе. С помощью диаграмм деятельности можно моделировать поведение физических и цифровых систем, а также бизнес-процессов. [22]

Диаграмма деятельности представлена на рисунке 10.

2.4.5 Диаграмма последовательности

Диаграмма последовательности используется для визуализации взаимодействия между объектами в системе. Она показывает порядок сообщений, которыми объекты обмениваются друг с другом во времени и их жизненный цикл. [23]

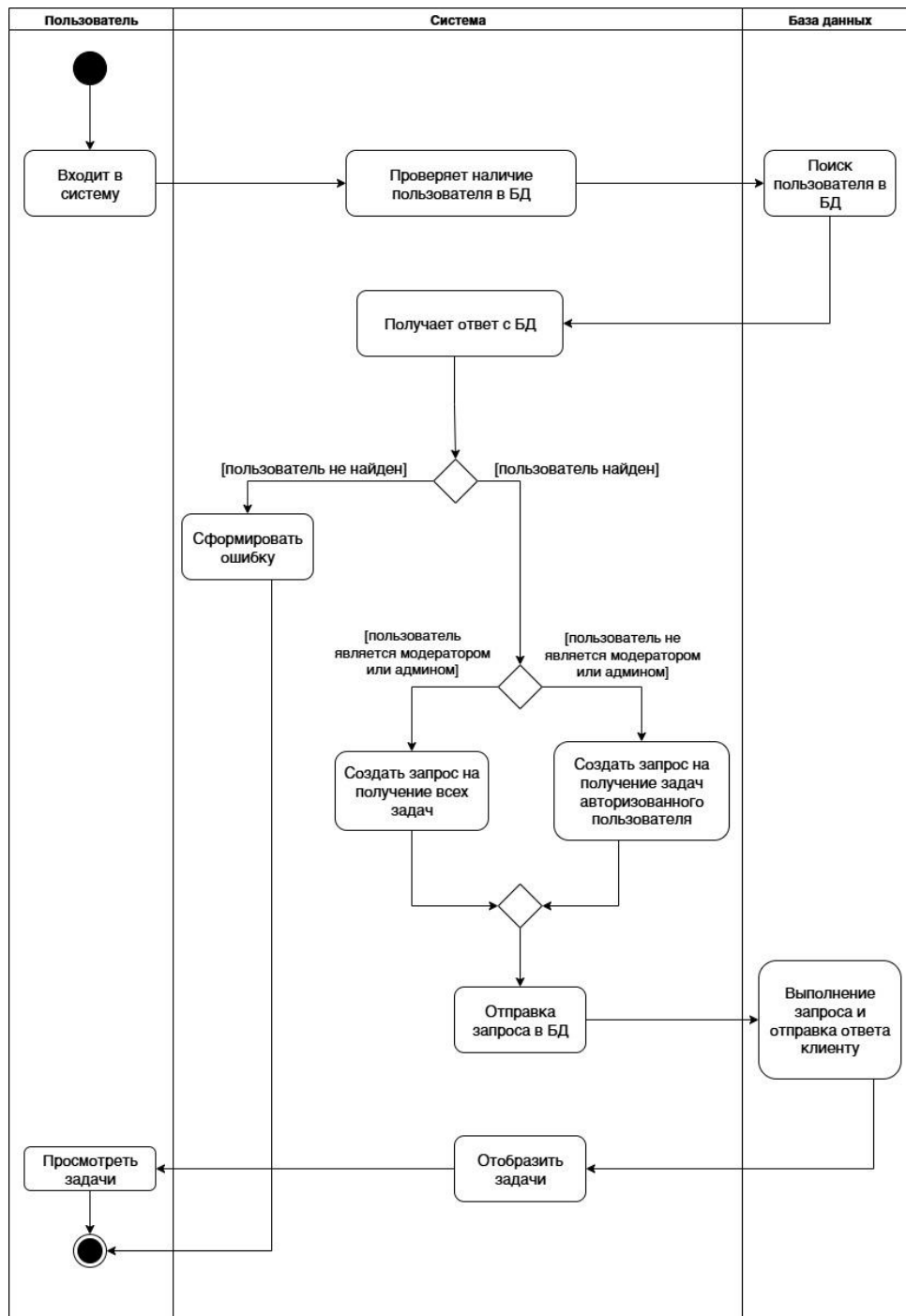


Рисунок 10 – Диаграмма деятельности

Диаграмма последовательности представлена на рисунке 11.

2.5 Структура базы данных

2.5.1 Логическая структура базы данных

Логическая структура базы данных – это концептуальный уровень организации данных, определяющий, как информация представлена, связана и обрабатывается внутри СУБД, независимо от того, где и как она физически хранится.

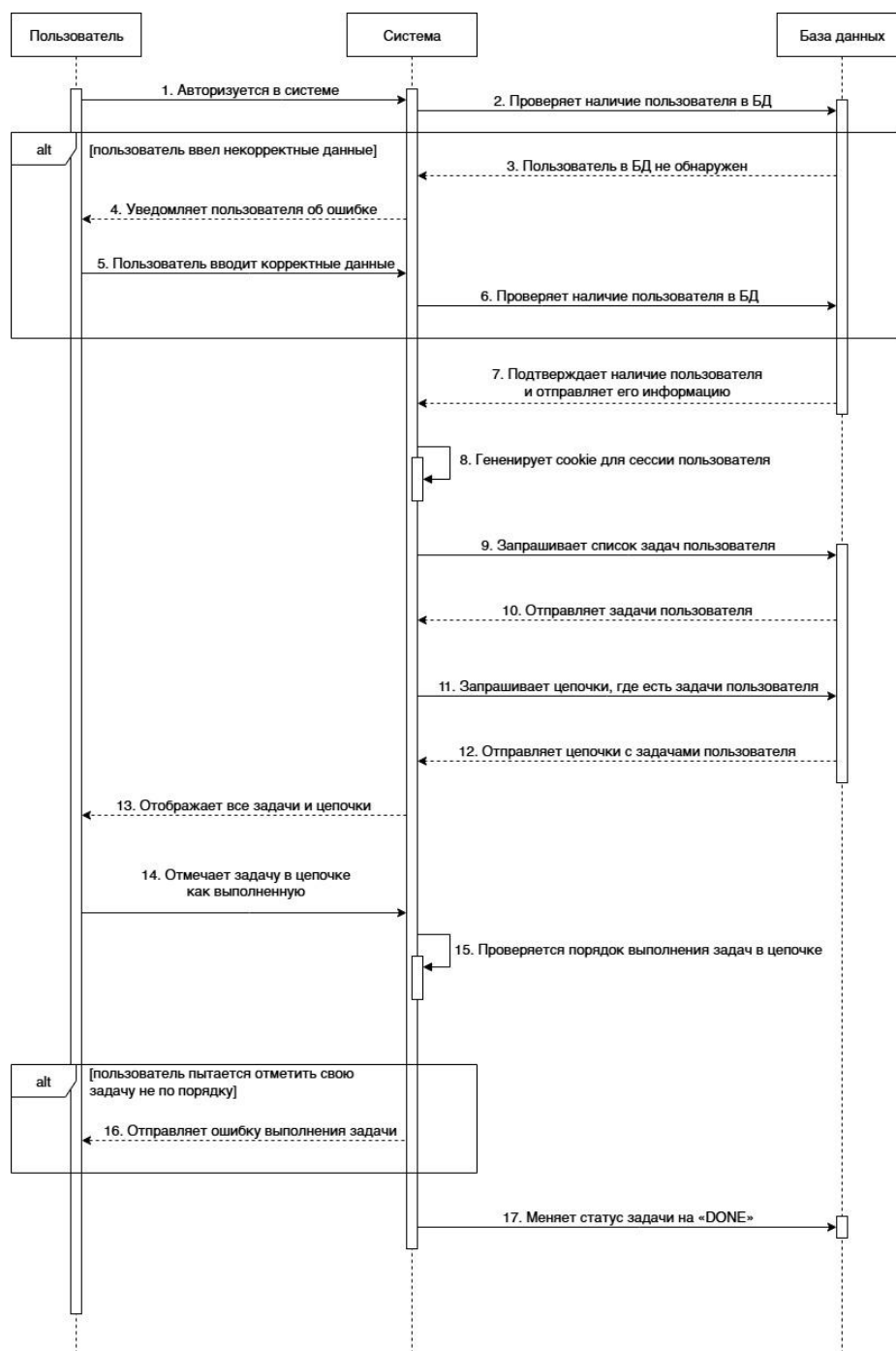


Рисунок 11 – Диаграмма последовательности

Логическая модель структуры базы данных представлена на рисунке 12. Она состоит из следующих сущностей: «Задача», «Цепочка задач», «Статус», «Пользователь», «Роли_пользователей», «Роль».

2.5.2 Физическая структура базы данных

Физическая структура базы данных – это нижний уровень организации, отражающий, как именно данные хранятся на физических носителях (дисках) и как СУБД (PostgreSQL) управляет операциями ввода-вывода.

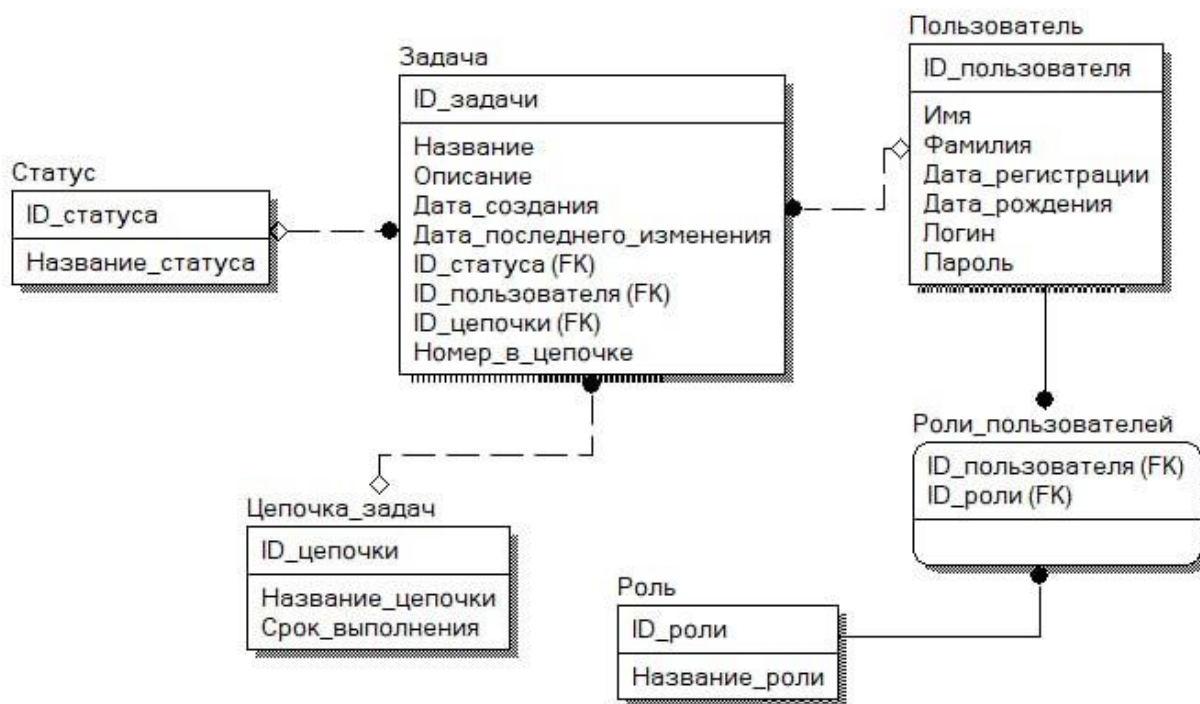


Рисунок 12 – Логическая модель базы данных

Физическая структура базы данных системы представлена на рисунке 13.

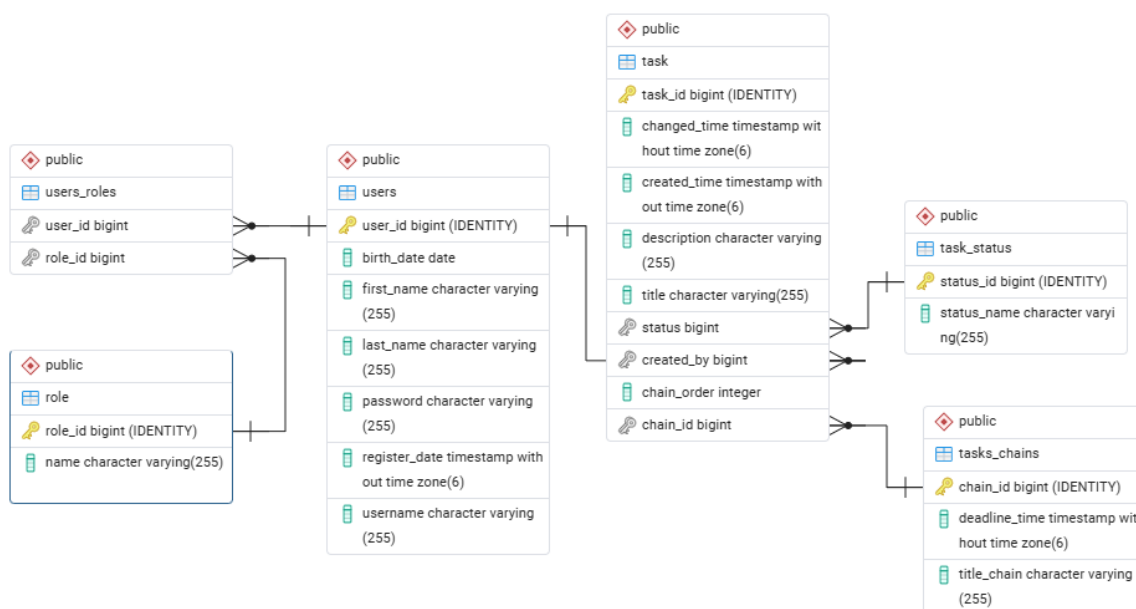


Рисунок 13 – Физическая модель базы данных

2.6 Описание основных алгоритмов системы

Алгоритм – это действия, которые позволяют решить определенную задачу. При этом они должны быть последовательными, то есть выполняться

в определённом порядке, чтобы получить нужный результат.

Все алгоритмы обладают следующими свойствами:

- конечность: алгоритм должен приводить к конкретному результату;
- предсказуемость: алгоритм при одинаковых условиях приводит к одинаковому результату;
- дискретность: алгоритм должен разделяться на более мелкие алгоритмы, каждый из которых должен приносить определённый результат;
- понятность: алгоритм должен быть описан так, чтобы его понял исполнитель;
- определенность: алгоритм должен быть чётким и однозначным. [24]

Основные схемы алгоритмов системы представлены на рисунках 14, 15, 16 и 17.

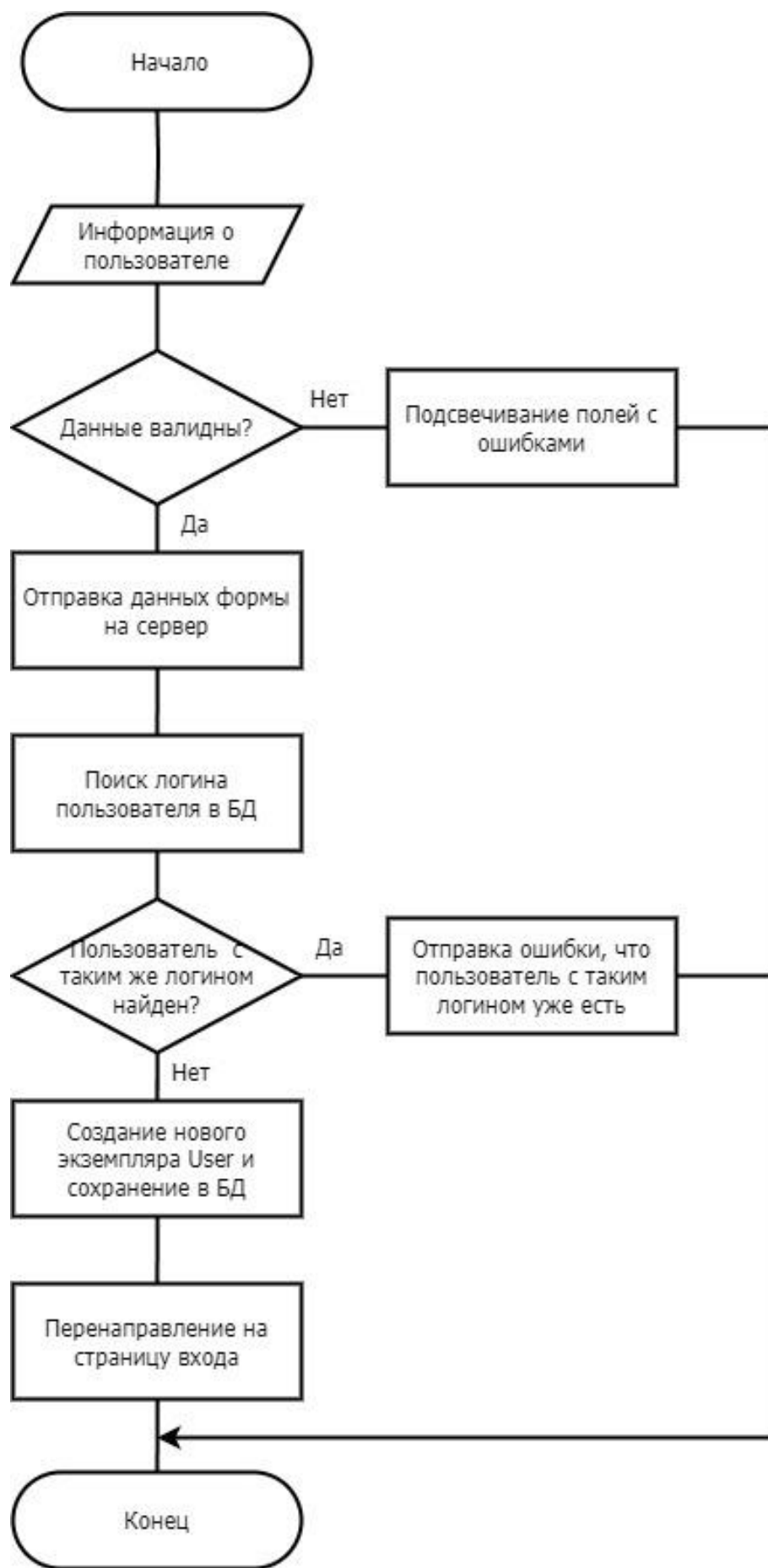


Рисунок 14 – Схема алгоритма регистрации пользователя

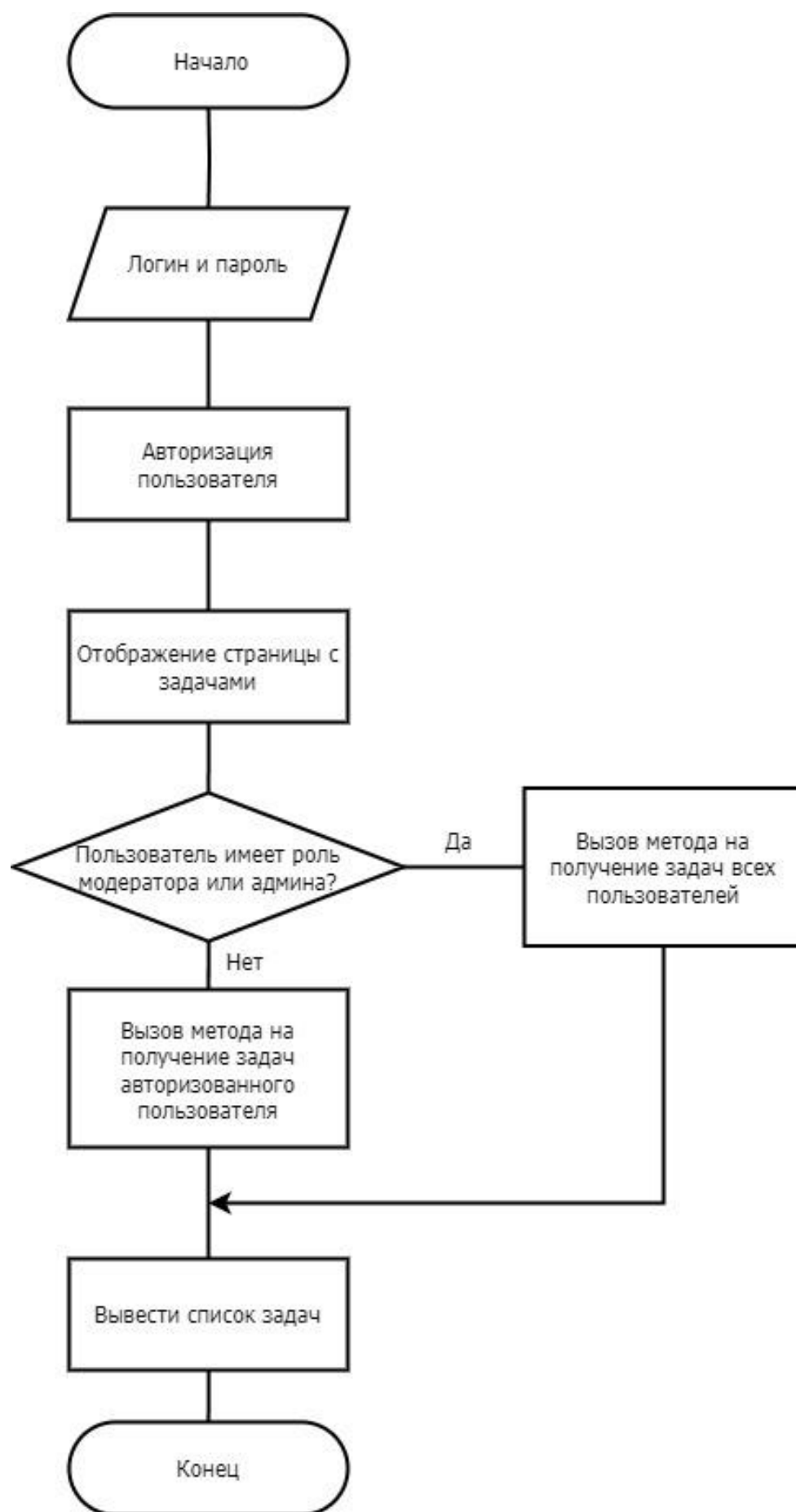


Рисунок 15 – Схема алгоритма получения списка задач

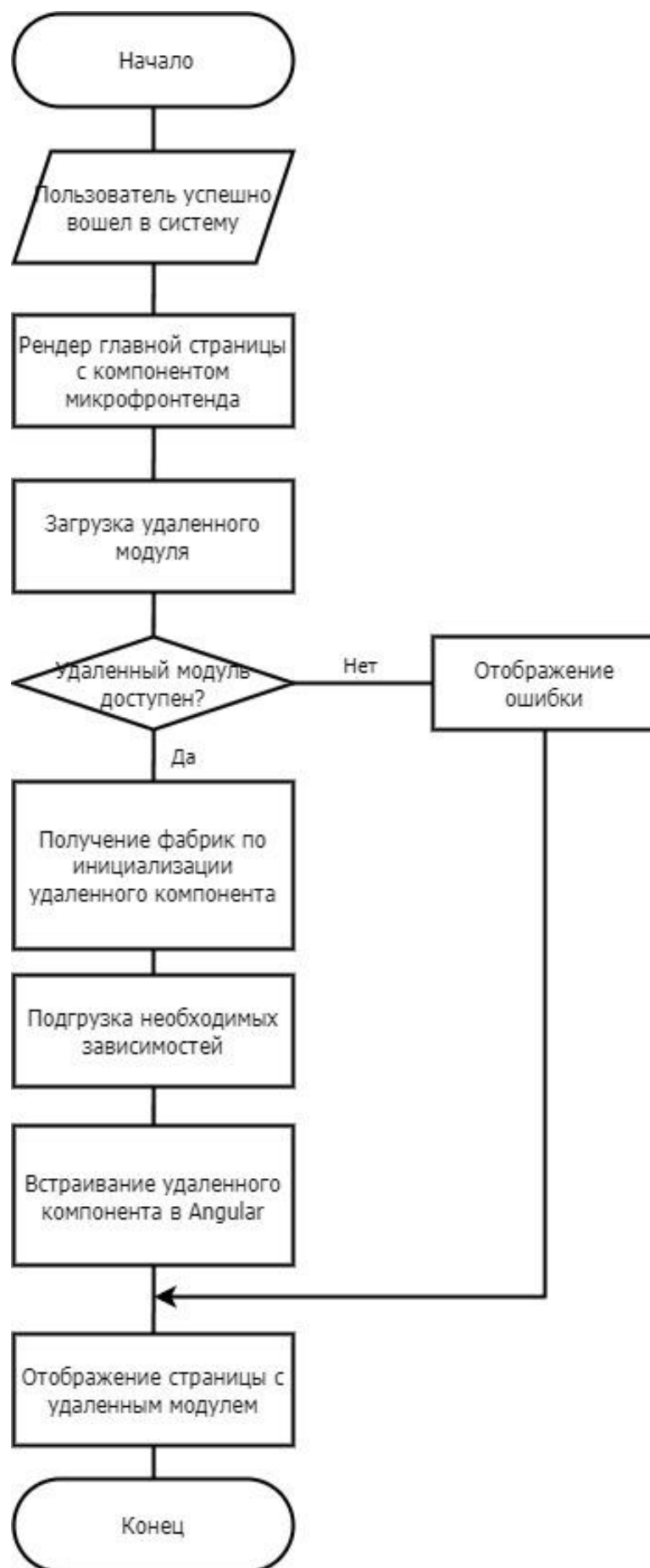


Рисунок 16 – Схема алгоритма загрузки микрофронтенда



Рисунок 17 – Схема алгоритма изменения цепочки задач на клиенте

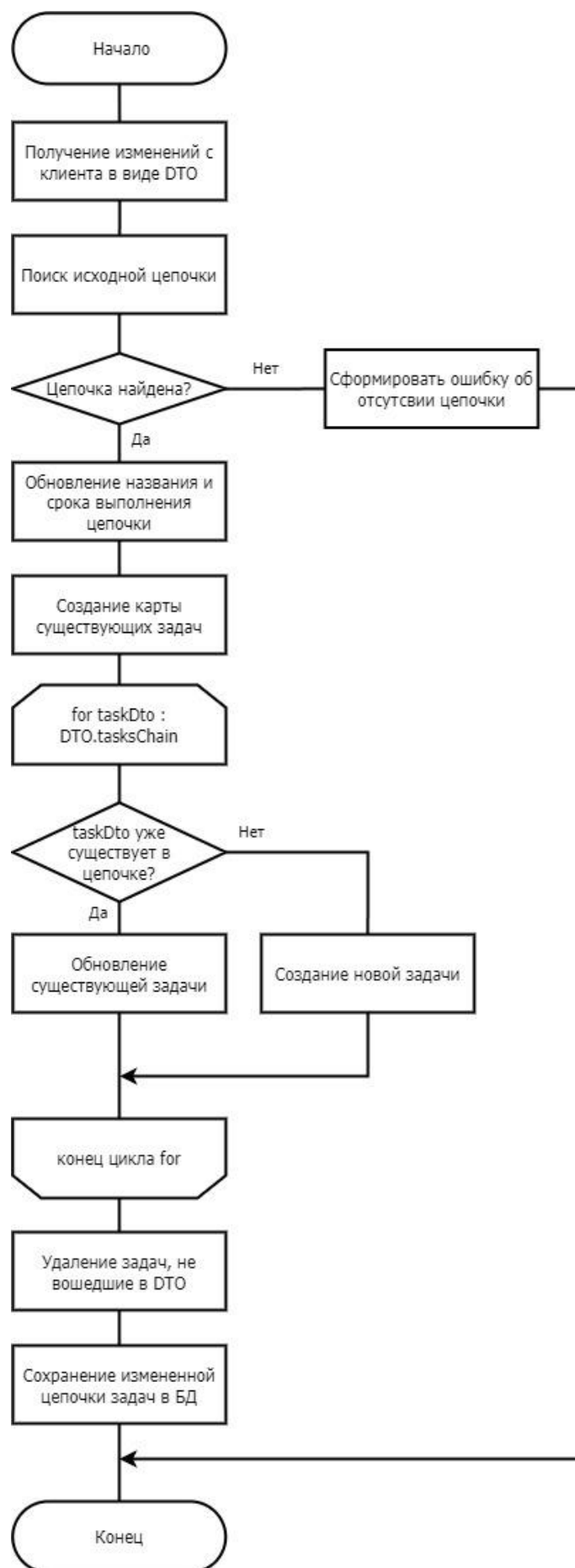


Рисунок 18 – Схема алгоритма изменения цепочки задач на сервере

3 Вычислительные эксперименты проверки эффективной программной реализации метода решения задачи

В рамках данного раздела осуществляется проверка функциональности и оценка эффективности внедрённой автоматизированной системы контроля исполнения задач. Задача экспериментальной части – подтвердить корректность работы ключевых компонентов: механизмов аутентификации и регистрации, инструментов создания и администрирования задач и их цепочек, а также подсистемы разграничения прав доступа на основе ролей.

В качестве контрольная примера рассмотрим процесс создания и изменения задачи, создание и изменение цепочки задач, а также проверку возможностей администратора и модератора.

При запуске приложения на экране появляется окно с формой для аутентификации. Форма представлена на рисунке 19.

Вход в систему

Введите данные для доступа к задачам

Логин

Введите логин

Пароль

Введите пароль

Войти

Нет аккаунта? [Зарегистрироваться](#)

Рисунок 19 – Форма входа в систему

Далее переходим по ссылке «Зарегистрироваться» под формой. После

перехода по ссылке, на экране появляется форма с регистрацией. Форма для регистрации представлена на рисунке 20.

Регистрация

Создайте аккаунт, чтобы начать работу

Имя

Фамилия

Логин

Пароль

Дата рождения

Уже есть аккаунт? [Войти](#)

Рисунок 20 – Форма регистрации

Заполним необходимые для регистрации поля. Пример заполненной формы представлен на рисунке 21.

Регистрация

Создайте аккаунт, чтобы начать работу

Имя: Михаил

Фамилия: Федулов

Логин: fedulov17

Пароль:

Дата рождения: 17.03.2005

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Рисунок 21 – Форма регистрации с заполненными полями

После нажатия по кнопке «Зарегистрироваться» успешно регистрируемся в системе. Приложение автоматически перенаправит на страницу входа в систему. Введем данные новосозданного пользователя в форму и нажмем на кнопку «Войти». Форма с заполненными полями представлена на рисунке 22.

Вход в систему

Введите данные для доступа к задачам

Логин

fedulov17

Пароль

.....

Войти

Нет аккаунта? [Зарегистрироваться](#)

Рисунок 22 – Форма входа с заполненными полями

После успешного входа на экране отобразится главный экран системы с боковой панелью. Боковая панель представляет собой микрофронтенд приложение, написанное с использованием React. На боковой панели находятся кнопки «Задачи» (для просмотра своих задач) и «Создать задачу» (для создания новой задачи), миниатюрный список последних измененных задач, блока с информацией об авторизованном пользователе. Главная страница представлена на рисунке 23.

Так как задач пока не создано, в центре экрана появилась кнопка «Создать задачу». Нажмем на нее и перейдем на форму создания задачи. Форма создания задачи представлена на рисунке 24.

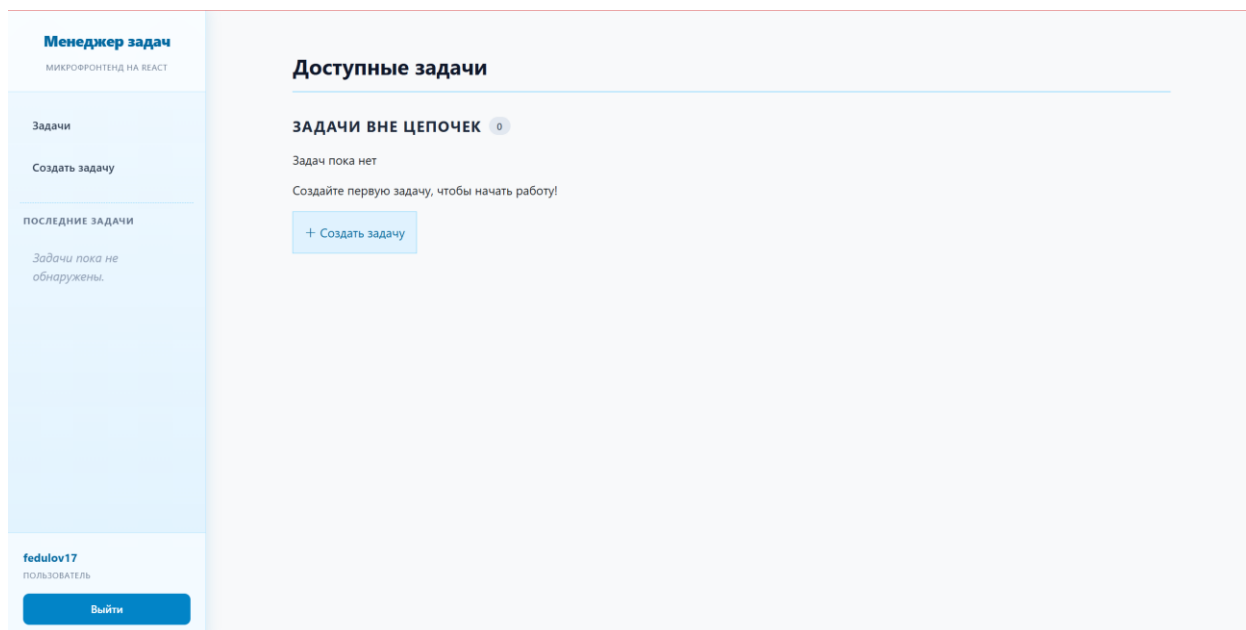
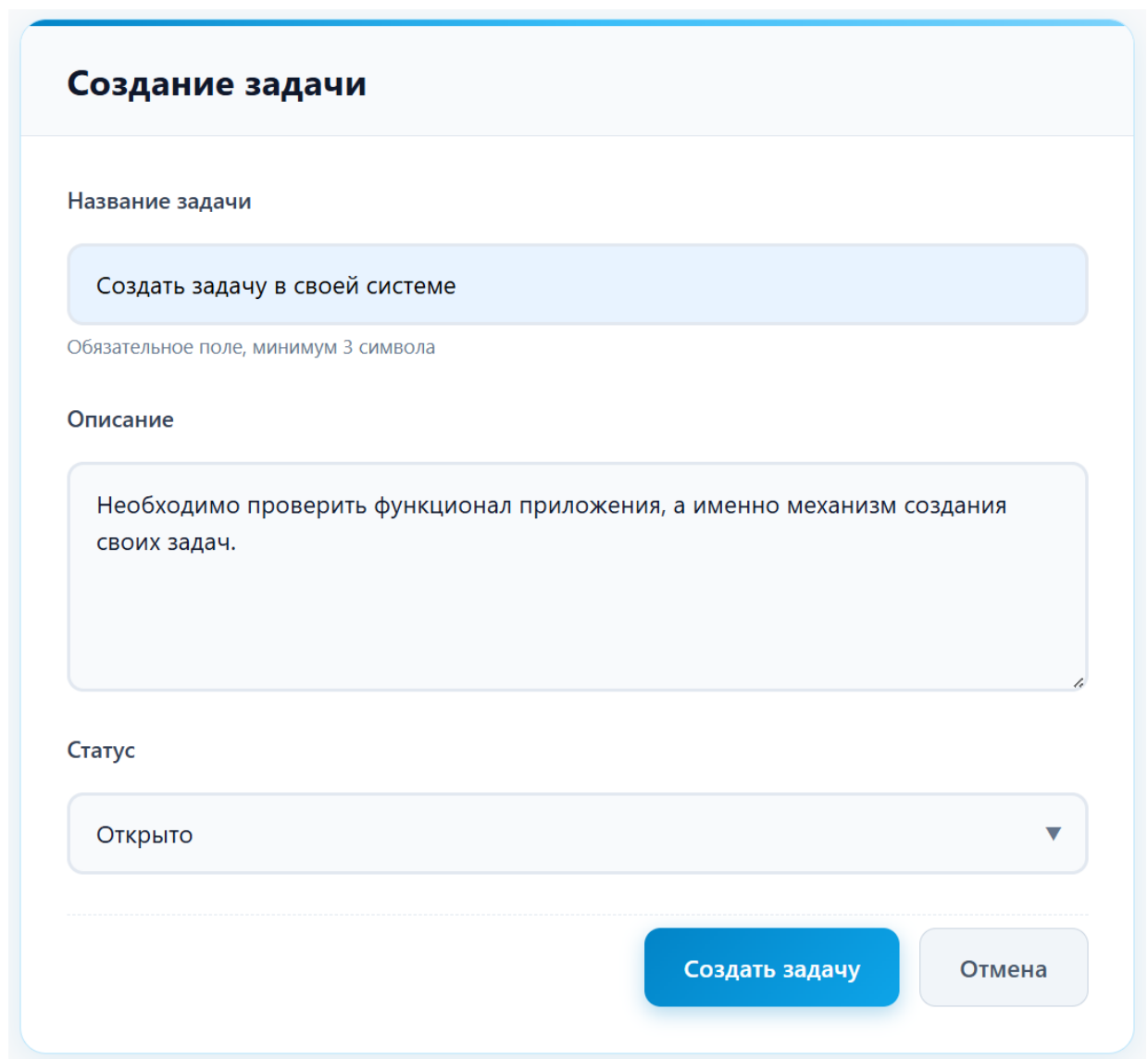


Рисунок 23 – Главная страница менеджера задач

The form is titled 'Создание задачи'. It has three main sections: 'Название задачи' with a text input field containing 'Например: Исправить баг в авторизации' and a note 'Обязательное поле, минимум 3 символа'; 'Описание' with a large text area containing 'Подробно опишите суть задачи, шаги воспроизведения или ожидаемый результат...'; and 'Статус' with a dropdown menu currently set to 'Открыто'. At the bottom right are two buttons: 'Создать задачу' and 'Отмена'.

Рисунок 24 – Форма создания задачи

Заполним все необходимые поля формы и нажмем на кнопку «Создать задачу». Форма с заполненными полями представлена на рисунке 25.



Создание задачи

Название задачи

Создать задачу в своей системе

Обязательное поле, минимум 3 символа

Описание

Необходимо проверить функционал приложения, а именно механизм создания своих задач.

Статус

Открыто

Создать задачу Отмена

Рисунок 25 – Форма с заполненными значениями

После нажатия по кнопке будет переход на страницу с задачами, где мы и можем посмотреть новую задачу. Главная страница с новой задачей представлена на рисунке 26.

В боковой панели мы также можем увидеть новую задачу. По нажатию по задаче в боковой панели, мы можем перейти на страницу изменения задачи. От имени пользователя мы можем изменить ее содержание, название и статус. Поменяем статус, описание и название задачи, для этого нажмем на кнопку «Редактировать» и изменим соответствующие значения. Форма изменения задачи с новыми значениями представлена на рисунке 27.

После изменения система перенаправит на страницу с задачами, где будет наша измененная задача.

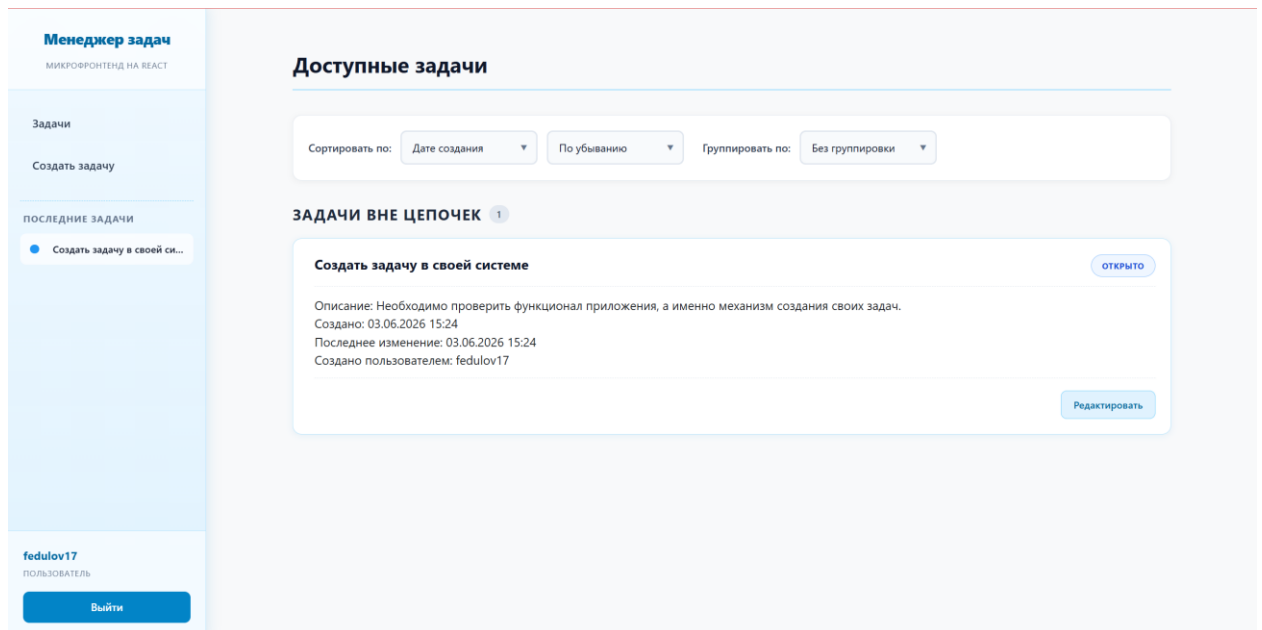


Рисунок 26 – Главная страница с созданной задачей

Рисунок 27 – Форма изменения задачи с новыми значениями

Главная страница с измененной задачей представлена на рисунке 28.

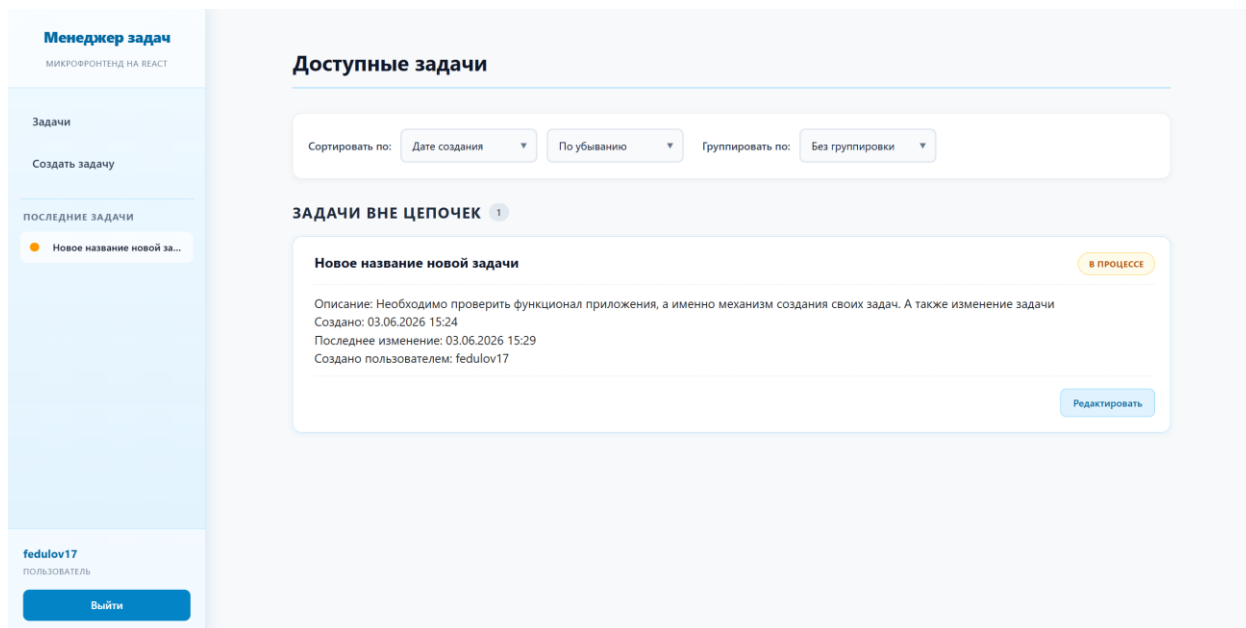


Рисунок 28 – Главная страница с измененной задачей

Теперь выйдем из профиля, нажав на кнопку «Выйти» в нижней части боковой панели и зайдем от имени администратора. На рисунке 29 представлена форма входа в систему с логином и паролем администратора.

Рисунок 29 – Форма с заполненными данными администратора

После входа в аккаунт администратора, на экране отобразится список задач всех пользователей. Главная страница администратора представлена на рисунке 30.

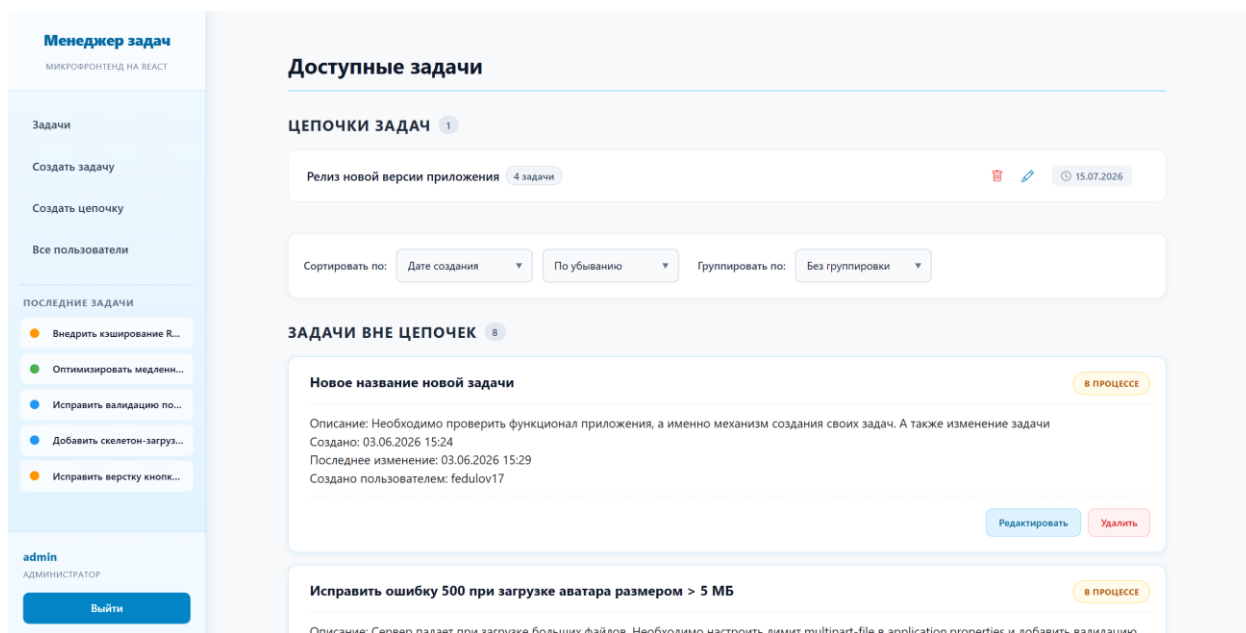


Рисунок 30 – Главная страница администратора

Для администратора и модераторов доступна возможность удаления как своих задач, так и задач других пользователей. Администратор может удалять пользователей вместе с их задачами, а также изменять пароли и назначать модераторов. В то же время, администратор и модератор являются такими же пользователями, что позволяет им создавать и изменять свои задачи.

Удалим одну из задач пользователей, для этого нажмем по кнопке «Удалить» первой задачи. Подтвердим действие и удалим задачу. Окно подтверждения представлено на рисунке 31.

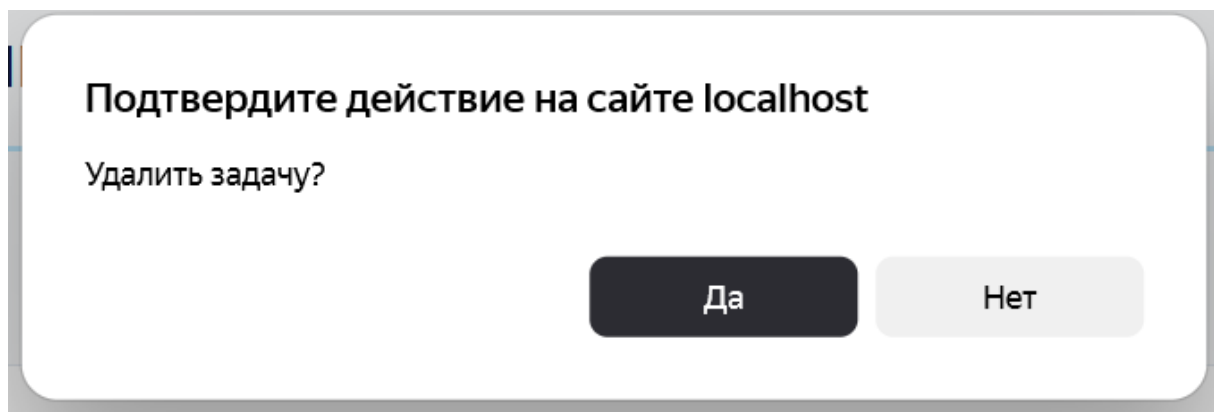


Рисунок 31 – Окно подтверждения удаления

После подтверждения действия страница обновится, и задача пропадет из списка. Обновленный список задач представлен на рисунке 32.

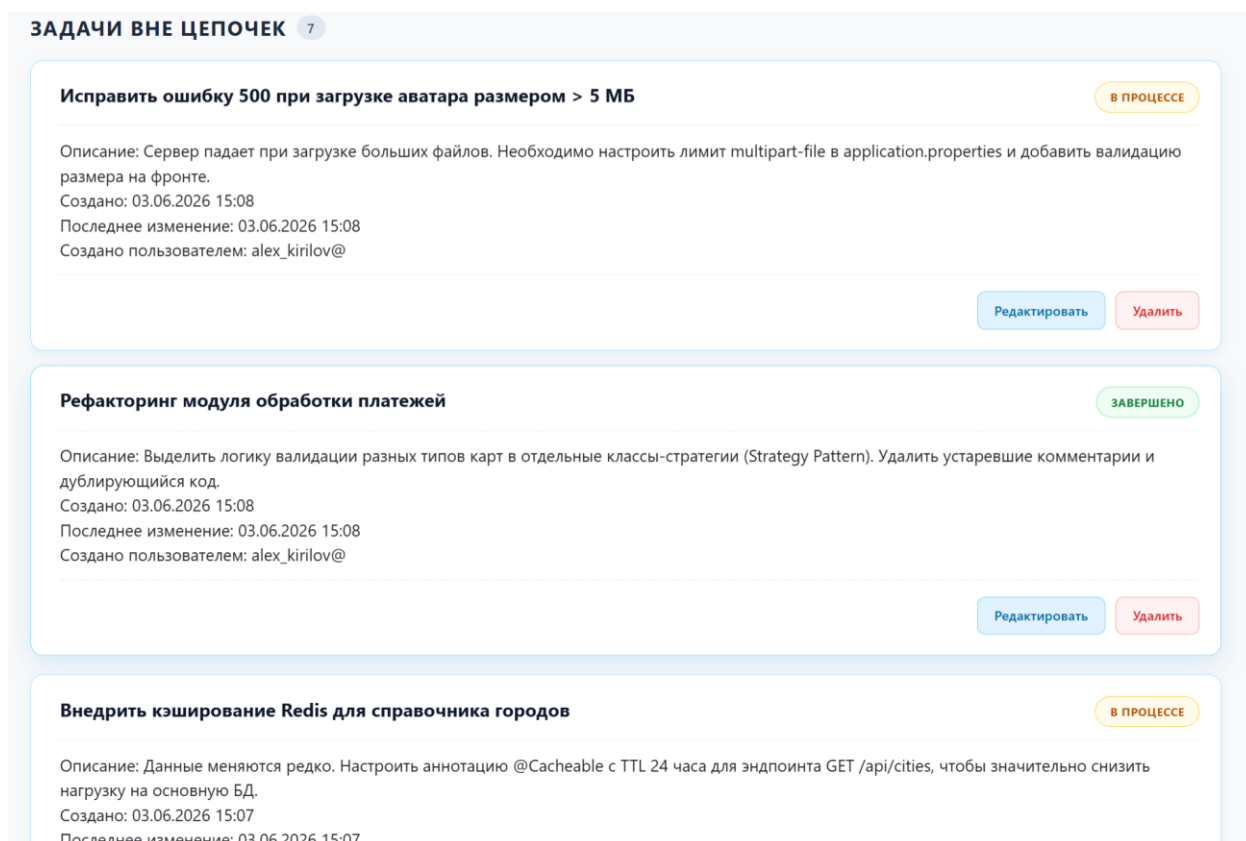


Рисунок 32 -Измененный список задач всех пользователей

Чтобы полностью убедиться, что одна единственная задача нового пользователя была удалена, воспользуемся функцией группировки задач по пользователям. Панель группировки и сортировки представлена на рисунке 33.

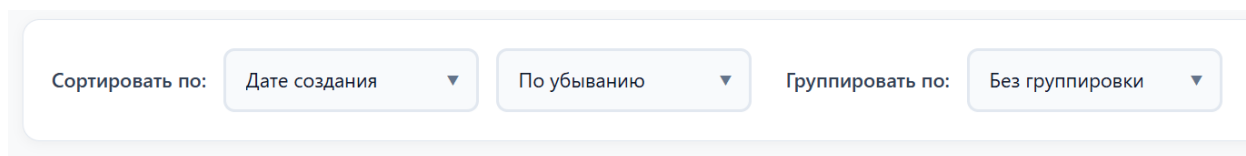


Рисунок 33 – Панель сортировки и группировки

Сгруппированный список задач представлен на рисунке 34.

Теперь попробуем от имени администратора назначить модератора и удалить пользователя. Для этого сначала нажмем по кнопке «Все пользователи» и перейдем на соответствующую страницу. Страница со всеми пользователями представлена на рисунке 34.

Назначим модератором недавно зарегистрированного пользователя,

Сортировать по:

Дате создания

По убыванию

Группировать по: Пользователю

ЗАДАЧИ ВНЕ ЦЕПОЧЕК

7

alex_kirilov@

(2)

Исправить ошибку 500 при загрузке аватара размером > 5 МБ

В ПРОЦЕССЕ

Описание: Сервер падает при загрузке больших файлов. Необходимо настроить лимит multipart-file в application.properties и добавить валидацию размера на фронте.
Создано: 03.06.2026 15:08
Последнее изменение: 03.06.2026 15:08
Создано пользователем: alex_kirilov@

Редактировать

Удалить

Рефакторинг модуля обработки платежей

ЗАВЕРШЕНО

Описание: Выделить логику валидации разных типов карт в отдельные классы-стратегии (Strategy Pattern). Удалить устаревшие комментарии и дублирующийся код.
Создано: 03.06.2026 15:08
Последнее изменение: 03.06.2026 15:08
Создано пользователем: alex_kirilov@

Редактировать

Удалить

petrov176

(2)

Внедрить кэширование Redis для справочника городов

В ПРОЦЕССЕ

Описание: Данные меняются редко. Настроить аннотацию @Cacheable с TTL 24 часа для эндпоинта GET /api/cities, чтобы значительно снизить нагрузку на основную БД.

Рисунок 34 – Сгруппированные задачи пользователей

Управление пользователями

ID пользователя в БД: 7

admin

Mike Smith

АДМИНИСТРАТОР

Изменить пароль

ID пользователя в БД: 17

ivanoff

Сергей Иванов

ПОЛЬЗОВАТЕЛЬ

Изменить пароль

Удалить пользователя

Назначить модератора

ID пользователя в БД: 18

petrov176

Петр Федоров

ПОЛЬЗОВАТЕЛЬ

Изменить пароль

Удалить пользователя

Назначить модератора

ID пользователя в БД: 19

alex_kirilov@

Александра Кириллова

Рисунок 35 – Страница со всеми пользователями

нажатием по кнопке «Назначить модератора» в его окне. После нажатия придет подтверждение действия в всплывающем окне, представленном на рисунке 36.

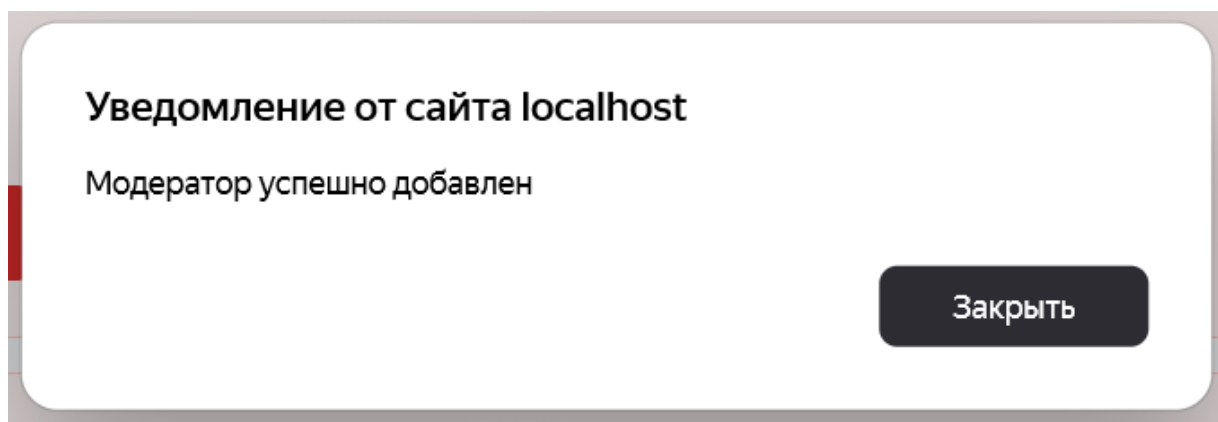


Рисунок 36 – Всплывающее окно назначения модератора

Проверим роли нового модератора: для этого выйдем из профиля администратора и зайдем в профиль нового пользователя fedulov17. После входа в профиль пользователя, на экране сразу появятся задачи и цепочки всех пользователей системы. На рисунке 37 представлена главная страница модератора.

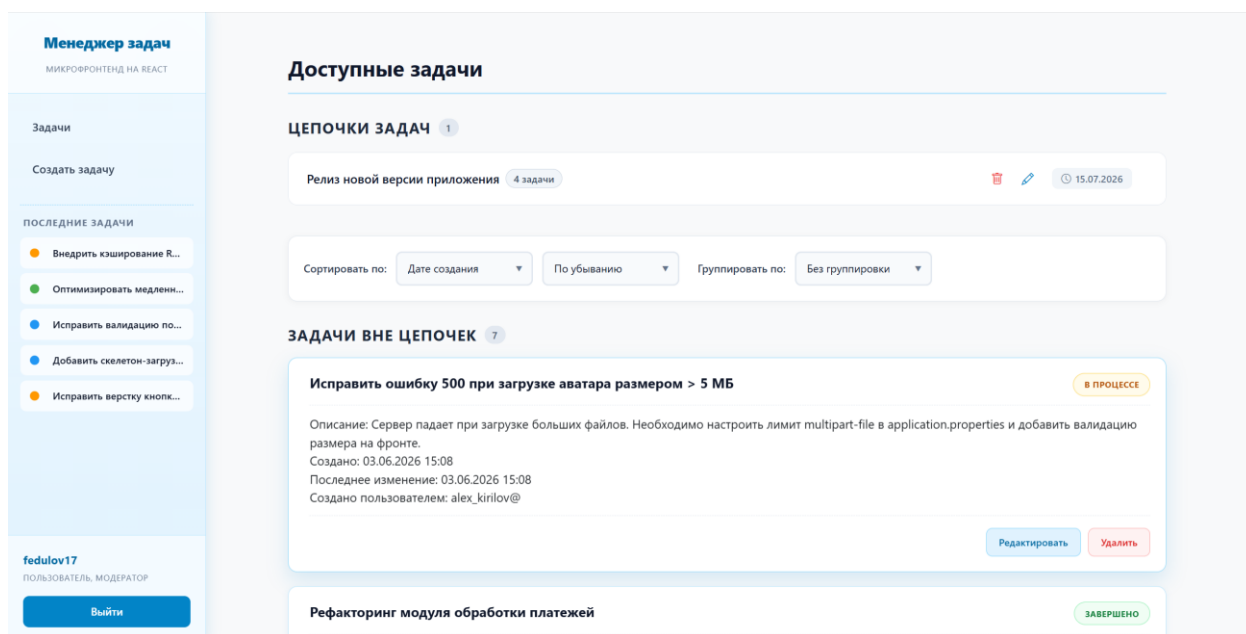


Рисунок 37 – Главная страница модератора

В нижнем левом углу страницы можно увидеть, что пользователь обладает ролями «Пользователь» и «Модератор». Отображаемые роли пользователя представлены на рисунке 38.

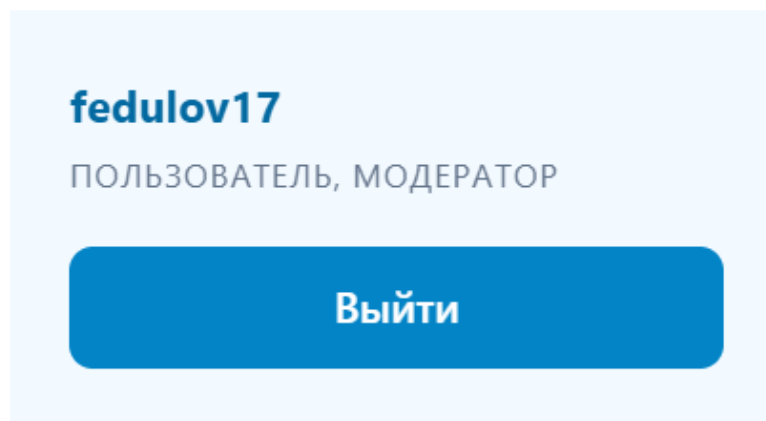


Рисунок 38 – Отображаемые роли авторизованного пользователя

Выйдем из профиля и зайдем обратно в профиль администратора. Теперь попробуем поменять пароль пользователя fedulov17. Нажмем на кнопку «Изменить пароль». После нажатия по кнопке вместо нее появятся текстовое поле и две кнопки, подтверждающее и отменяющее действие, соответственно. На рисунке 39 представлен этот элемент.

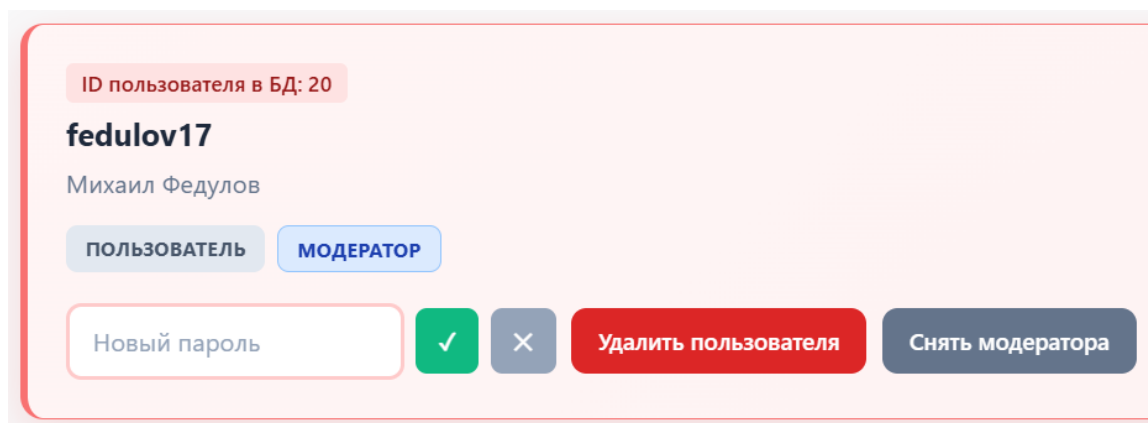


Рисунок 39 – Изменение пароля пользователя

Зададим новый пароль (например, «theNewPassword») и подтвердим действие. В подтверждение действия появится всплывающее окно, представленное на рисунке 40.

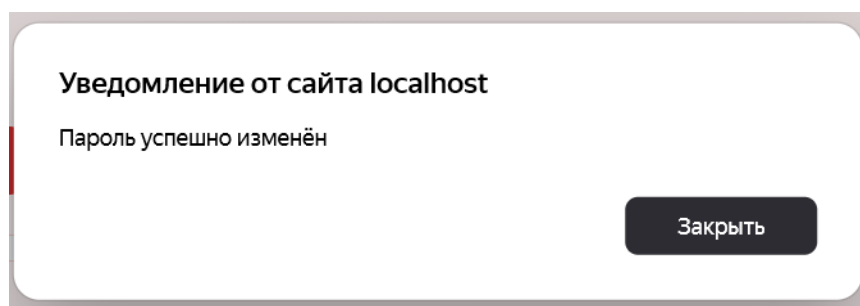


Рисунок 40 – Окно подтверждения смены пароля

Теперь пользователь может заходить в свой профиль с новым паролем. Теперь попробуем удалить пользователя, например, с логином «alex_kirilov@». Нажмем на кнопку «Удалить пользователя» в окне пользователя и подтвердим действие в всплывающем окне, представленном на рисунке 41.

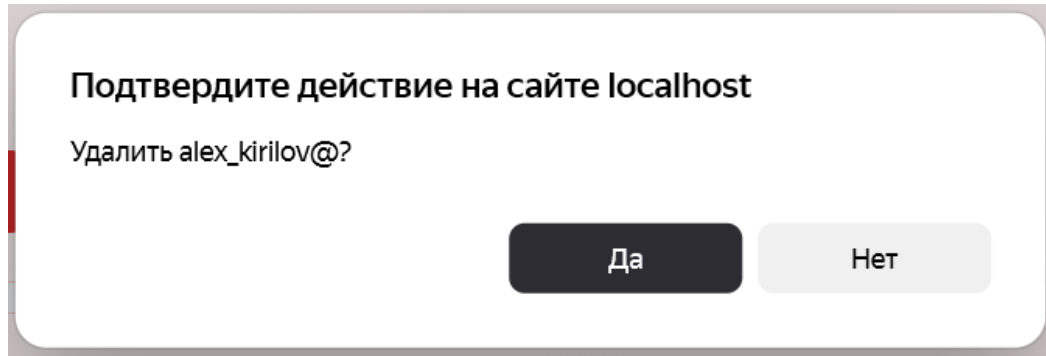


Рисунок 41 – Окно подтверждения удаления пользователя

С удалением пользователя удаляются и его задачи, проверим это на странице всех задач. Список задач представлен на рисунке 42.

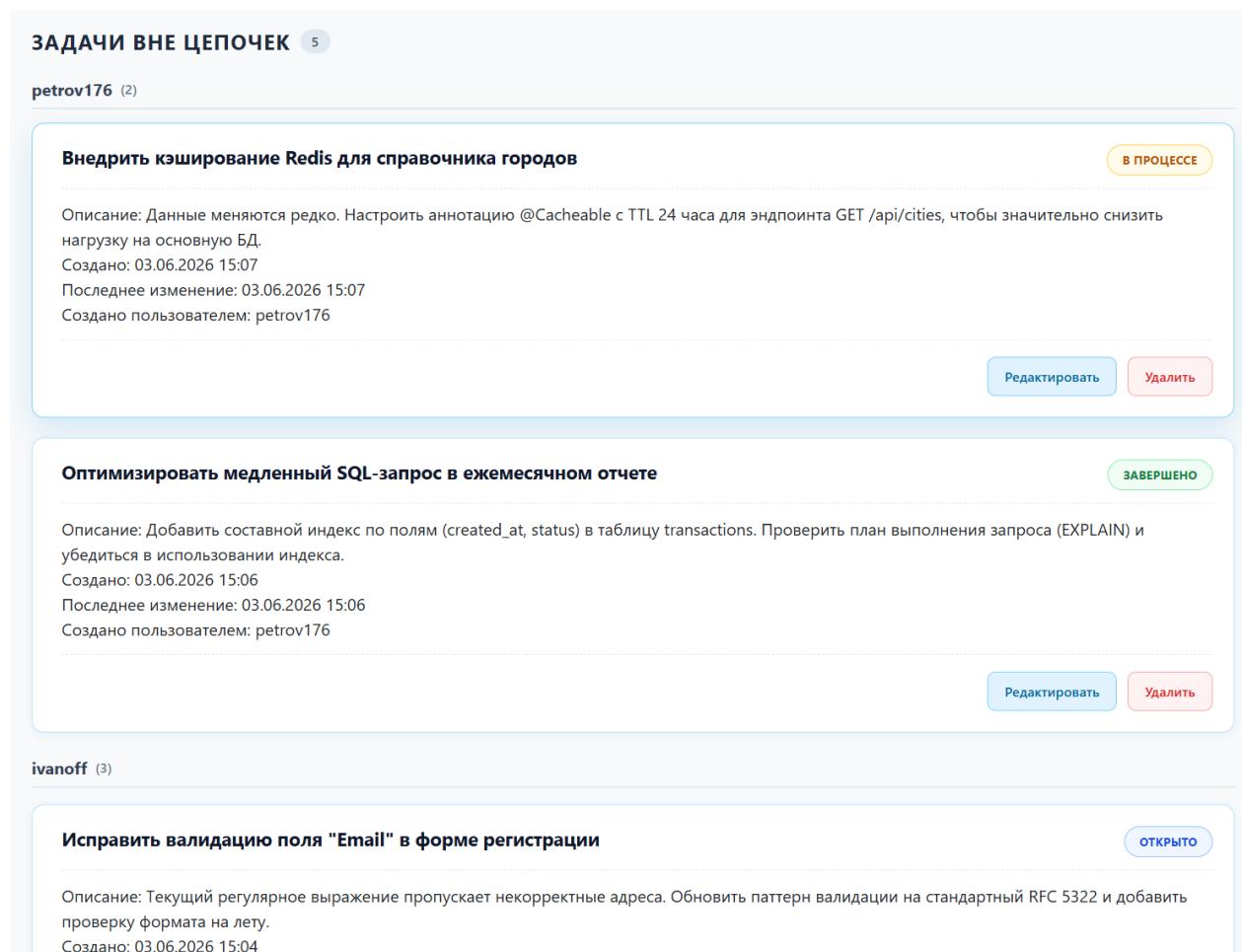


Рисунок 42 – Список задач без удаленного пользователя

Далее проверим механизм организации цепочек задач. Такие цепочки могут создавать и изменять только модераторы и администратор, поэтому авторизуемся как администратор и нажимаем на кнопку в боковой панели «Создать цепочку». Страница с созданием цепочки представлена на рисунке 43.

Рисунок 43 – Страница с созданием цепочки

Здесь можно задать название цепочке и ее срок выполнения. Чтобы добавлять новые задачи, необходимо нажимать на кнопку «Добавить задачу». Интерфейс обладает возможностью перетаскивать карточки с задачами и менять их последовательность в цепочке. Заполним форму и назначим пользователей на задачи. Заполненная цепочка представлена на рисунке 44.

После нажатия по кнопке «Создать цепочку» приложение перенаправит на страницу со всеми задачами и цепочками. Новая цепочка задач представлена на рисунке 45.

Цепочки задач можно изменять, например, в случае внезапной новой задачи или допущенной ошибки, а также переназначать пользователей. Попробуем изменить цепочку и поменяем местами первую и вторую задачу местами. Для изменения нажмем на кнопку и перейдем на страницу редактирования цепочки. Перетаскиванием поменяем местами задачи и сохраним изменения.

Создание цепочки задач

ИНФОРМАЦИЯ О ЦЕПОЧКЕ

Название цепочки *

Повышение безопасности

Дедлайн

10.06.2026 15:30

ЗАДАЧИ В ЦЕПОЧКЕ

+ Добавить задачу

ШАГ 1

Название задачи *

Устранить уязвимость SQL-инъекции в модуле поиска

Пользователь *

petrov176

Описание

Заменить опасную конкатенацию строк в DAO-слое на использование параметризованных запросов (PreparedStatement или JPA Criteria API).

ШАГ 2

Название задачи *

Написать unit-тесты для UserService

Пользователь *

ivanoff

Описание

Покрыть методы регистрации и аутентификации тестами с использованием Mockito и JUnit. Достичь покрытия кода (code coverage) не менее 80%.

ШАГ 3

Название задачи *

Провести нагрузочное тестирование эндпоинта авторизации

Пользователь *

fedulov17

Описание

Использовать JMeter для эмуляции 1000 одновременных запросов в секунду. Зафиксировать метрики времени отклика и потребления памяти сервером.

Отмена

Создать цепочку

Рисунок 44 – Новая цепочка задач

Доступные задачи

ЦЕПОЧКИ ЗАДАЧ 2

Релиз новой версии приложения 3 задачи

15.07.2026

Повышение безопасности 3 задачи

10.06.2026

1.Устранить уязвимость SQL-инъекции в модуле поиска

2.Написать unit-тесты для UserService

3.Провести нагрузочное тестирование эндпоинта авторизации

Рисунок 45 – Новая цепочка задач в общем списке

Измененная цепочка задач представлена на рисунке 46.

50

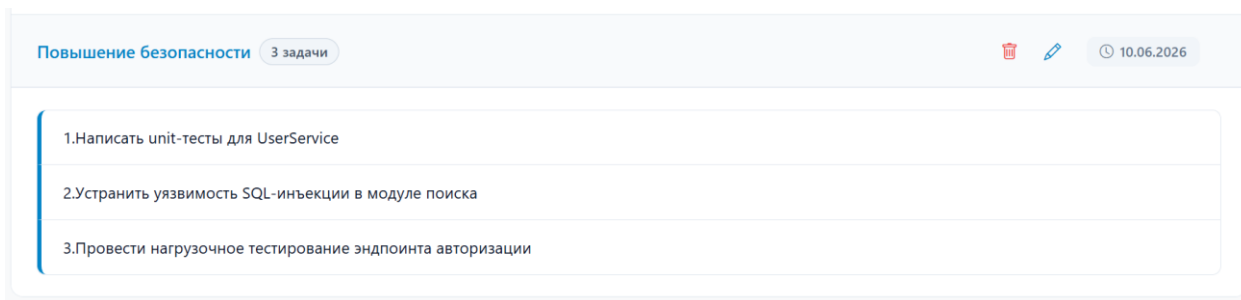


Рисунок 46 – Измененная цепочка задач

При нажатии на задачу из цепочки, откроется модальное окно с информацией о задаче и кнопками управления: «Удалить» и «Изменить» (только для администратора и модераторов), а также «Закрыть» (для закрытия модального окна). Модальное окно представлено на рисунке 47.

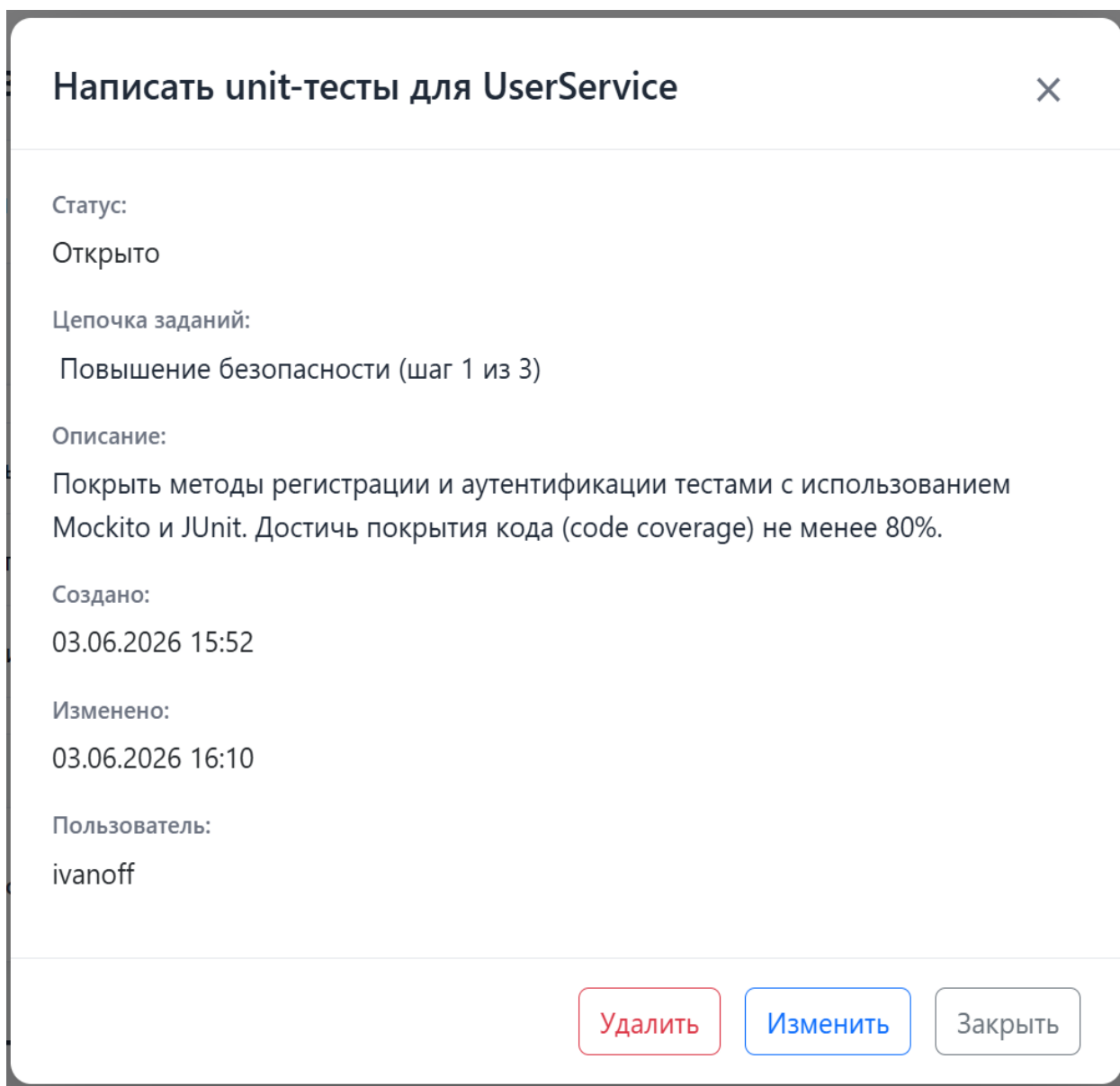


Рисунок 47 – Модальное окно с информацией о задаче

Попробуем выполнить цепочки задач. Задачи в цепочках могут подтверждать только пользователи, для кого они были предназначены. Зайдем от имени пользователя «ivanoff» и подтвердим выполнение. Когда мы нажмем на задачу в цепочке, в модальном окне появится кнопка подтверждения выполнения задачи. Модальное окно с подтверждением представлено на рисунке 48.

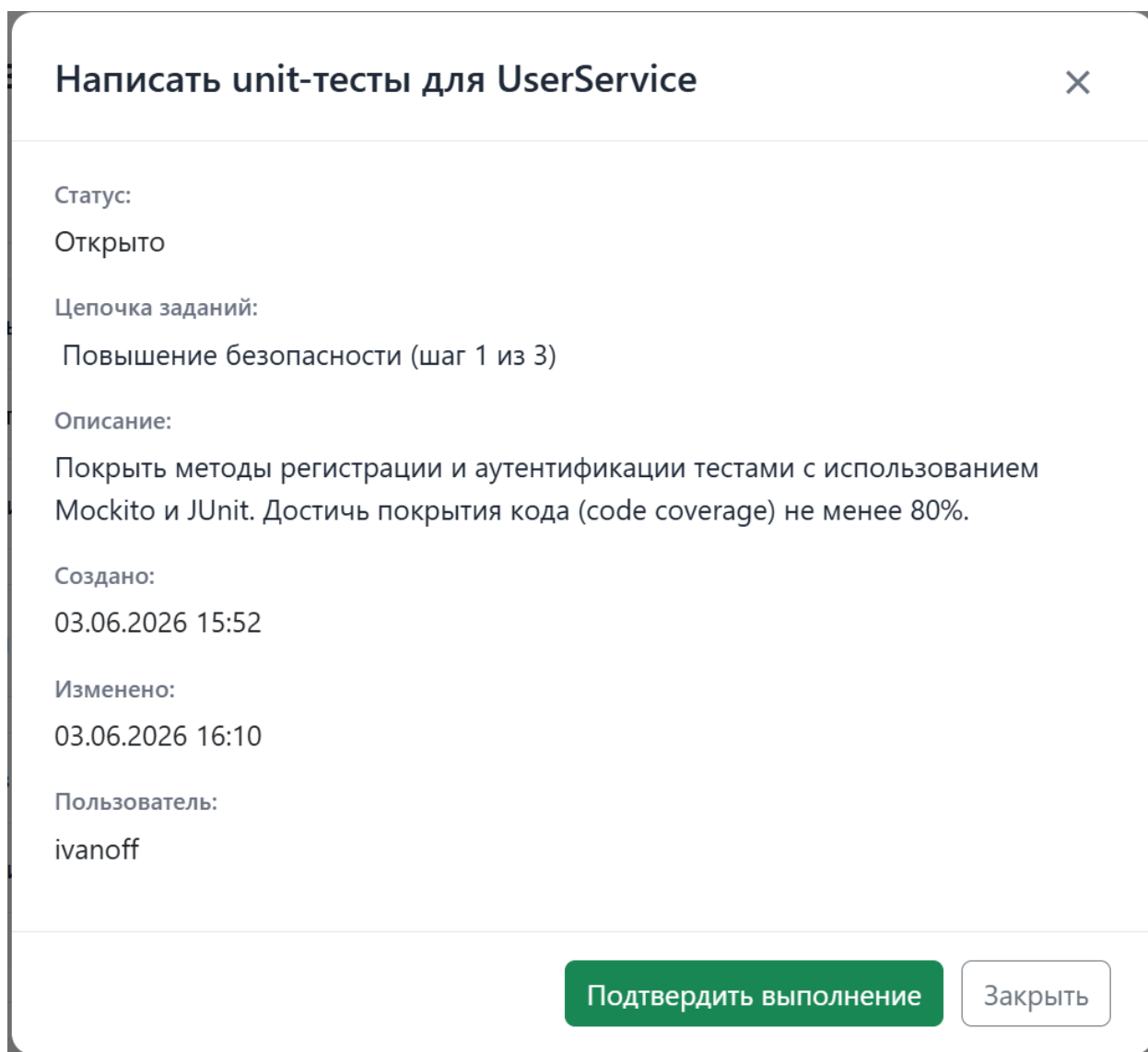


Рисунок 48 – Модальное окно с подтверждением

Когда кнопка будет нажата, статус задачи изменится на «Завершено», а строка с этой задачей окрасится в зеленый цвет. Так как цепочка задач требует строгого порядка выполнения задач, в системе реализовано ограничение выполнения задачи, если предыдущие не были выполнены. Например, в данной цепочке, следующие пользователи должны выполнять задачи в

строгом порядке: «ivanoff», «petrov176», «fedulov17». Попробуем выполнить последнюю задачу без предыдущих. Зайдем в профиль от лица «fedulov17» и откроем окно со своей задачей. В модальном окне задачи будет отсутствовать кнопка подтверждения и будет отображаться плашка, с информацией о предыдущей задаче. Измененное модальное окно представлено на рисунке 49.

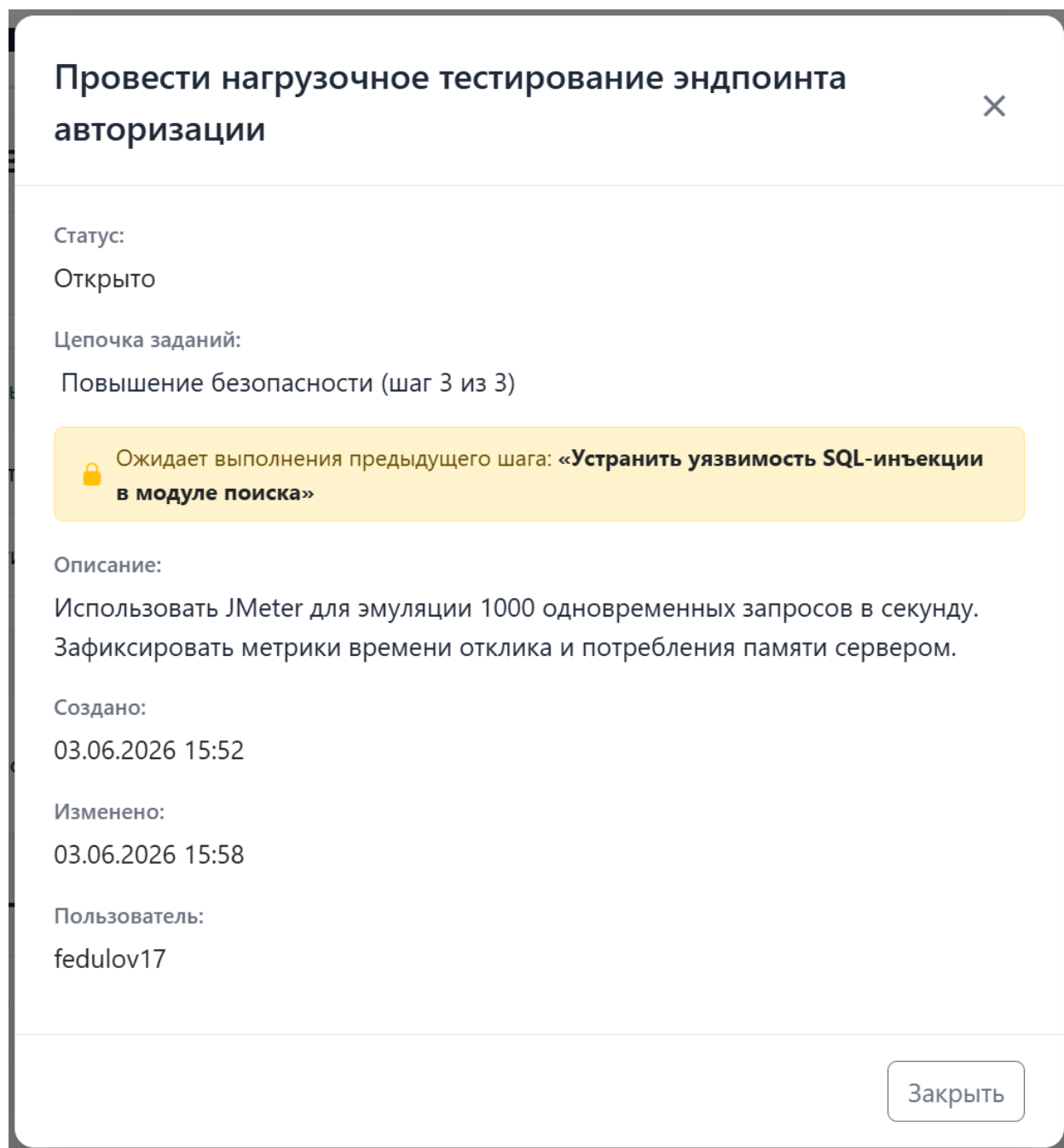


Рисунок 49 – Измененное модальное окно с задачей

После выполнения всех задач в цепочке, она окрасится в зеленый цвет. Выполненная цепочка задач представлена на рисунке 50.

ЦЕПОЧКИ ЗАДАЧ 1



Повышение безопасности

3 задачи

1. Написать unit-тесты для UserService

2. Устранить уязвимость SQL-инъекции в модуле поиска

3. Провести нагрузочное тестирование эндпоинта авторизации

Рисунок 50 – Выполненная цепочка задач

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана автоматизированная информационная система по отслеживанию выполнения задач в компании. Система отвечает всем требованиям задания и реализует основные функции:

- регистрация и аутентификация пользователей в системе;
- создание, изменение и удаление задач пользователей;
- создание и контроль цепочек задач;
- разграничение прав доступа к компонентам системам посредством ролевой модели;
- назначение модераторских прав доступа обычным пользователям;
- отказоустойчивость и надежность системы за счет использования микрофронтендов на клиентской части.

Разработан логический проект на языке UML с использованием онлайн-сервиса draw.io.

Разработаны логическая модель базы данных в среде AllFusion Erwin Data Modeler и физическая модель базы данных в СУБД PostgreSQL

Система разработана на языках программирования Java и TypeScript с использованием Spring Framework, Angular Framework, React, Module Federation.

Были использованы среды разработки OpenIDE и VS Code.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 About Trello [Электронный ресурс]. – URL: <https://trello.com/about> (дата обращения: 01.04.2026).

2 Jira: что это такое за система управления проектами [Электронный ресурс]. – URL: <https://blog.skillfactory.ru/glossary/jira/> (дата обращения: 01.04.2026).

3 Asana: обзор инструмента для управления проектами и задачами [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/asana-dlya-upravleniya-proektami/> (дата обращения: 01.04.2026).

4 UML: что за язык моделирования, преимущества использования - типы UML-диаграмм, как их создать, примеры [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/uml-diagrammy/#cto-takoe> (дата обращения: 01.04.2026).

5 CASE-средства [Электронный ресурс]. – URL: <https://ai.mitup.ru/journal/doklad/case-sredstva/> (дата обращения: 01.04.2026).

6 AllFusion Modeling Suite [Электронный ресурс]. – URL: <https://compress.ru/article.aspx?id=11790> (дата обращения: 01.04.2026).

7 Что такое Java и каково ее применение? [Электронный ресурс]. – URL: https://www.java.com/ru/download/help/whatis_java.html (дата обращения: 01.04.2026).

8 TypeScript: как с ним работать и чем он отличается от JavaScript [Электронный ресурс]. – URL: <https://blog.skillfactory.ru/typescript-kak-s-nim-rabotat/> (дата обращения: 01.04.2026).

9 OpenIDE – бесплатная IDE для Java, Python, Go и TypeScript [Электронный ресурс]. – URL: <https://openide.ru/> (дата обращения: 01.04.2026).

10 Что такое Visual Studio Code? [Электронный ресурс]. – URL: <https://ru.hexlet.io/blog/posts/cto-takoe-visual-studio-code> (дата обращения: 01.04.2026).

11 Windows 10 – Википедия [Электронный ресурс]. – URL: https://ru.wikipedia.org/wiki/Windows_10 (дата обращения: 01.04.2026).

12 Spring Framework: что это и зачем нужен - принципы и структура фреймворка, как начать пользоваться Java Spring для начинающих [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/framework-spring-java/> (дата обращения: 01.04.2026).

13 Introduction to Spring Data JDBC [Электронный ресурс]. – URL: <https://www.geeksforgeeks.org/advance-java/introduction-to-spring-data-jdbc/> (дата обращения: 01.04.2026).

14 Angular: что это за фреймворк, преимущества - основы работы с Angular [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/framevork-angular/> (дата обращения: 01.04.2026).

15 React [Электронный ресурс]. – URL: <https://blog.skillfactory.ru/glossary/react/> (дата обращения: 01.04.2026).

16 Webpack Module Federation: «официальное» решение в микрофронтендах [Электронный ресурс]. – URL: <https://habr.com/ru/companies/alfa/articles/668118/> (дата обращения: 01.04.2026).

17 PostgreSQL: что это за СУБД, основы и преимущества [Электронный ресурс]. – URL: <https://blog.skillfactory.ru/glossary/postgresql/> (дата обращения: 01.04.2026).

18 Веб-приложение: что это такое, как работает, методы обеспечения безопасности [Электронный ресурс]. – URL: https://rt-solar.ru/products/solar_appscreener/blog/4628/ (дата обращения: 01.04.2026).

19 Фронтенд или бэкенд: по какому пути в разработке пойти [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/chem-otlichaetsya-backend-i-frontend/> (дата обращения: 01.04.2026).

20 Использование диаграммы вариантов использования UML при проектировании программного обеспечения [Электронный ресурс]. – URL: <https://habr.com/ru/articles/566218/> (дата обращения: 01.04.2026).

21 Unified Modeling Language [Электронный ресурс]. – URL: <https://studfile.net/preview/10903376/page:10/> (дата обращения: 01.04.2026).

22 UML activity diagrams [Электронный ресурс]. – URL: <https://www.drawio.com/docs/diagram-types/uml/activity-diagrams/> (дата обращения: 01.04.2026).

23 Диаграмма последовательности (sequence-диаграмма) [Электронный ресурс]. – URL: <https://habr.com/ru/articles/814769/> (дата обращения: 01.04.2026).

24 Что такое алгоритмы и какими они бывают [Электронный ресурс]. – URL: <https://practicum.yandex.ru/blog/algoritmy-i-kakimi-oni-byvayut/> (дата обращения: 01.04.2026).

ПРИЛОЖЕНИЕ А

Руководство пользователя

А.1 Назначение системы

Автоматизированная информационная система по отслеживанию выполнения задач в компании «Менеджер задач» предназначена для хранения и управления задачами пользователей в рамках небольшой компании или команды. Система упрощает планирование заданий и ускоряет разработку различных проектов.

А.2 Условия работы системы

Требования к техническому обеспечению:

- монитор с разрешающей способностью не ниже 1920 x 1080;
- манипуляторы – мышь, клавиатура.
- тип операционной системы – Windows 10 и выше;
- виртуальная машина – Java Virtual Machine 17;
- Java Development Kit 17;
- среда выполнения JavaScript-кода Node.js;
- СУБД PostgreSQL 17.

А.3 Установка приложения

Серверная часть приложения предоставляется в виде jar-архива, который можно запустить с помощью любого сервера, поддерживающего Java-приложения (например, Tomcat), или IDE (например, OpenIDE).

Для запуска клиентской части приложения необходимо выполнить команды «npm install» и «npm run start:all» в командной строке из директории проекта. После выполнения последней команды откроется браузер с страницей по пути <http://localhost:4200>. Если браузер не открылся, следует самостоятельно открыть его и ввести в адресную строку ссылку: <http://localhost:4200>.

А.4 Руководство пользователя

В браузере откроется страница входа в систему (рисунок А.1).

Вход в систему

Введите данные для доступа к задачам

Логин

Введите логин

Пароль

Введите пароль

Войти

Нет аккаунта? [Зарегистрироваться](#)

Рисунок А.1 – Форма входа в систему

Если пользователь еще не регистрировался в системе, то это можно сделать по ссылке «Зарегистрироваться» в нижней части формы. После нажатия по ссылке, перед пользователем откроется страница с формой регистрацией (рисунок А.2).

Для дальнейшей регистрации необходимо заполнить следующие поля: имя, фамилия, логин, пароль, дата рождения (рисунок А.3). Далее необходимо подтвердить регистрацию и нажать на кнопку «Зарегистрироваться». В случае успеха, пользователя перенаправит на страницу входа.

Регистрация

Создайте аккаунт, чтобы начать работу

Имя

Иван

Фамилия

Иванов

Логин

Придумайте логин

Пароль

Минимум 6 символов

Дата рождения

ДД.ММ.ГГГГ



Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Рисунок А.2 – Форма регистрации

Регистрация

Создайте аккаунт, чтобы начать работу

Имя

Михаил

Фамилия

Федулов

Логин

fedulov17

Пароль

.....

Дата рождения

17.03.2005

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

Рисунок А.3 – Заполненная форма регистрации

Пользователь вводит свои данные и нажимает на кнопку «Войти» (рисунок А.4).

Вход в систему

Введите данные для доступа к задачам

Логин

fedulov17

Пароль

.....

Войти

Нет аккаунта? [Зарегистрироваться](#)

Рисунок А.4 – Заполненная форма входа

Далее отображается главная страница системы (рисунок А.5).

Менеджер задач
МИКРОФРОНТЕНД НА REACT

Задачи

Создать задачу

ПОСЛЕДНИЕ ЗАДАЧИ

Задачи пока не обнаружены.

fedulov17
ПОЛЬЗОВАТЕЛЬ

Выйти

Доступные задачи

ЗАДАЧИ ВНЕ ЦЕПОЧЕК 0

Задач пока нет

Создайте первую задачу, чтобы начать работу!

+ Создать задачу

Рисунок А.5 – Главная страница

На главной странице представлена боковая панель, которая предоставляет функционал по перемещению между страницами приложения (рисунок А.6). Верхняя часть боковой панели является кликабельной и по нажатию по ней можно перейти на главную страницу. Далее представлены кнопки «Задачи» (для отображения своих задач) и «Создать задачу» (для отображения формы создания задачи). В разделе «Последние задачи» будут отображаться последние добавленные задачи пользователя. В нижней части боковой панели пользователь может просмотреть свой логин и свои роли, а также нажать на кнопку «Выход», которая совершит выход из системы и перенаправит на страницу входа.

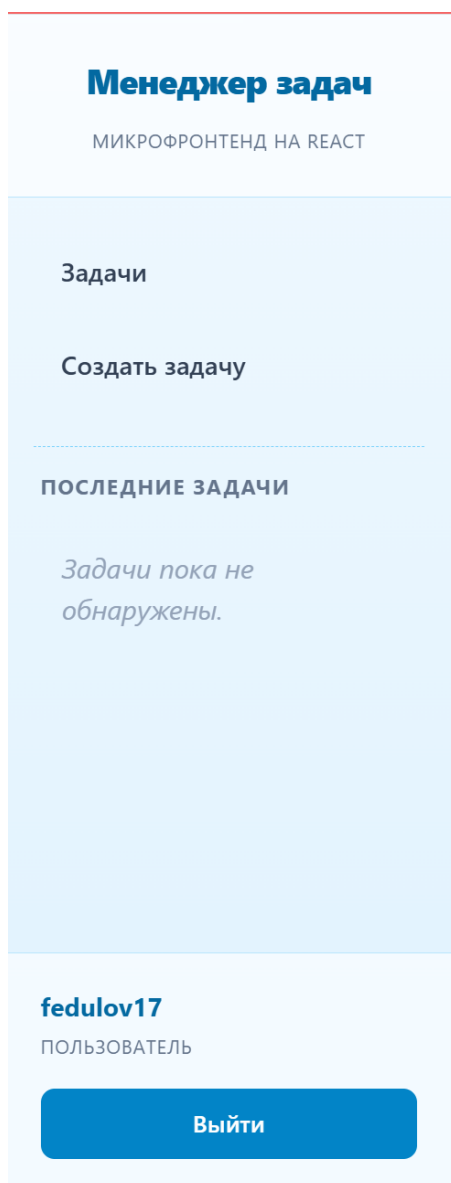


Рисунок А.6 – Боковая панель приложения

Для создания задачи необходимо нажать на кнопку создать задачу в центре экрана (если задач нет) или на кнопку в боковой панели. После нажатия откроется форма создания задачи (рисунок А.7).

Рисунок А.7 – Форма создания задачи

Заполним все необходимые поля и нажмем на кнопку «Создать задачу» (рисунок А.8). После создания задачи произойдет переход на страницу с задачами (рисунок А.9). Название, описание и статус задачи можно изменить по нажатию кнопки «Редактировать».

В системе реализованы административные права: чтобы воспользоваться ими, необходимо зайти в систему с логином и паролем «admin». Административные права позволяют управлять задачами всех пользователей и всеми профилями пользователей системы. Также можно назначать модераторов, которые смогут удалять как свои задачи, так и задачи

других пользователей. Управление пользователями представлено на рисунке А.10.

Создание задачи

Название задачи

Проверка работоспособности проекта

Обязательное поле, минимум 3 символа

Описание

Необходимо проверить проект, написать тесты, добавить документацию.

Статус

В процессе

Создать задачу

Отмена

Рисунок А.8 – Заполненная форма создания задачи

Доступные задачи

Сортировать по:

Дате создания

По убыванию

Группировать по:

Без группировки

ЗАДАЧИ ВНЕ ЦЕПОЧЕК

1

Проверка работоспособности проекта

ОТКРЫТО

Описание: Необходимо проверить проект, написать тесты, добавить документацию.

Создано: 03.06.2026 16:21

Последнее изменение: 03.06.2026 16:21

Создано пользователем: fedulov17

Редактировать

Рисунок А.9 – Новая задача

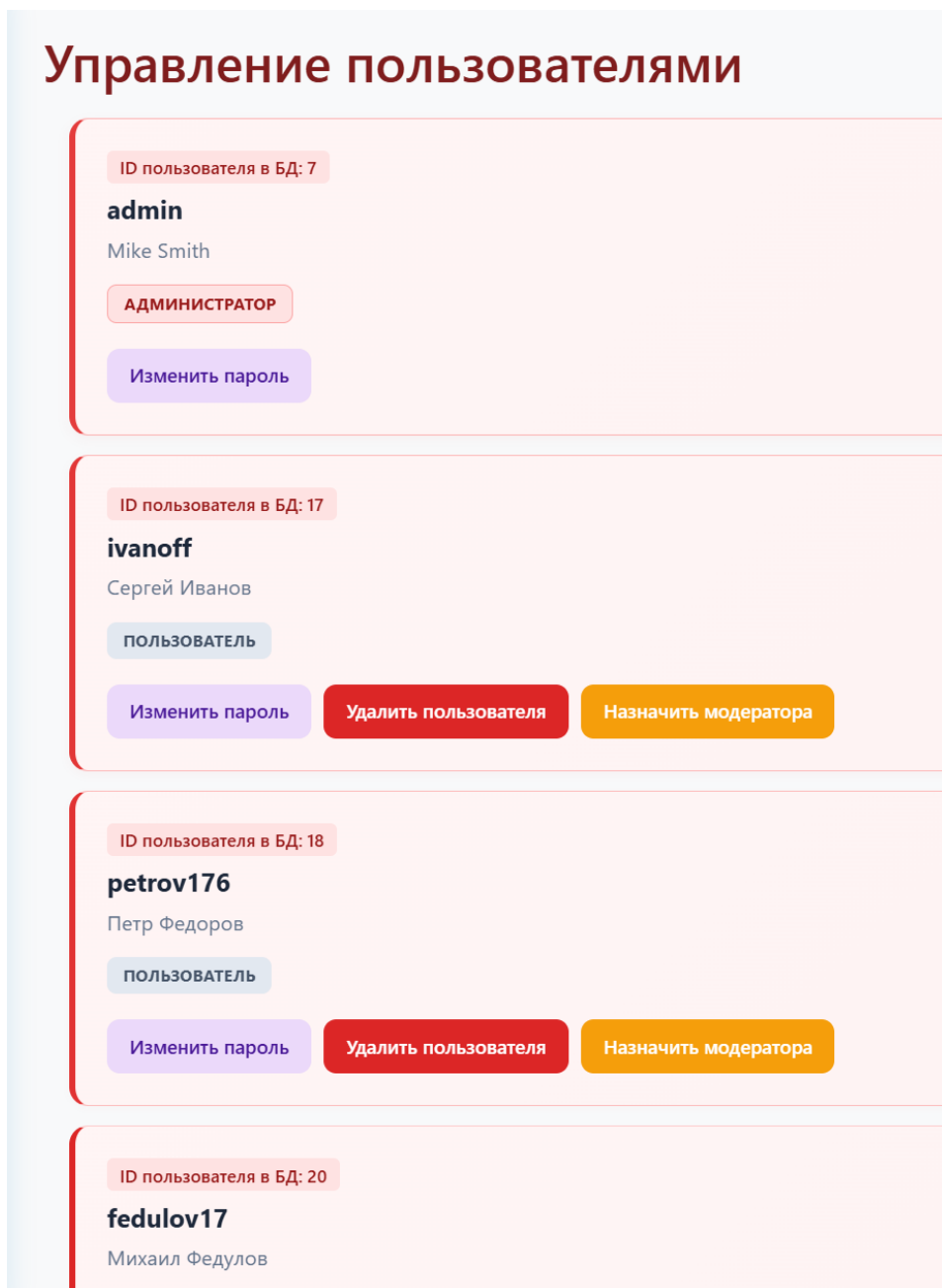


Рисунок А.10 – Страница с управлением пользователями системы

ПРИЛОЖЕНИЕ Б

Листинг приложения

Классы сущностей системы:

```
@Entity
@Table(name = "users")
public class User implements UserDetails {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "user_id")
    private long id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "register_date")
    private LocalDateTime registerDate;

    @Column(name = "birth_date")
    private LocalDate birthDate;

    private String username;

    private String password;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "users_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"))
    private List<Role> roles;

    @OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
    private List<Task> tasks = new ArrayList<>();

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
```

```

        return getRoles();
    }
}

@Entity
@RequiredArgsConstructor
@Table(name = "task")
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "task_id")
    private long id;

    private String title;

    private String description;

    @Column(name = "created_time")
    private LocalDateTime createdTime;

    @Column(name = "changed_time")
    private LocalDateTime changedTime;

    @ManyToOne
    @JoinColumn(name = "status")
    private TaskStatus status;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "created_by")
    @OnDelete(action = OnDeleteAction.CASCADE)
    private User user;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "chain_id")
    private TasksChain chain;

    @Column(name = "chain_order")
    private Integer chainOrder;
}

```

```

@Entity
@Table(name = "task_status")
public class TaskStatus{
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "status_id")
    private long id;

    @Column(name = "status_name")
    private String statusName;
}

```

```

@Entity
@Table(name = "role")
public class Role implements GrantedAuthority {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "role_id")
    private long id;

    private String name;
    @ManyToMany(mappedBy = "roles")
    private List<User> users;

    @Override
    public @Nullable String getAuthority() {
        return getName();
    }
}

```

```

@Entity
@RequiredArgsConstructor
@Table(name = "tasks_chains")
public class TasksChain {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "chain_id")
    private long id;
}

```

```

        private String titleChain;

        private LocalDateTime deadlineTime;

        @OneToMany(mappedBy = "chain", cascade = CascadeType.ALL,
orphanRemoval = true, fetch = FetchType.LAZY)
        @OrderBy("chainOrder ASC")
        private List<Task> tasksChain = new ArrayList<>();

        public void addTask(Task task){
            tasksChain.add(task);
            task.setChain(this);
        }
    }
}

```

Классы DTO:

@Getter

@Setter

```

public class RoleDto {
    private long id;
    private String name;
}

```

@Getter

@Setter

```

public class TaskDto {
    private long id;
    private String title;
    private String description;
    private LocalDateTime created_time;
    private LocalDateTime modified_time;
    private String status;
    private String created_by;
}

```

@Getter

@Setter

```

public class UserDto {

```

```

    private long id;
    private String firstName;
    private String lastName;
    private LocalDateTime registerDate;
    private LocalDate birthDate;
    private String username;
    private String password;
    private List<RoleDto> roles;
}

```

```

@Getter
@Setter
public class UserRoleDto {
    private String username;
    private String role;
}

```

```

@Getter
@Setter
public class TasksChainDto {
    private long id;
    private String titleChain;
    private LocalDateTime deadlineTime;
    private List<TaskDto> tasksChain;
}

```

Контроллеры серверной части:

```

@AllArgsConstructor
@RestController
@RequestMapping("/tasks")
public class TaskController {

```

```

    private final TaskService taskService;

```

```

    @GetMapping("/{id}")
    public ResponseEntity<Optional<TaskDto>>
getTaskById(@PathVariable(name = "id") Long id){
        Optional<TaskDto> foundedTask = taskService.getTaskById(id);
        if(foundedTask.isPresent()) return
ResponseEntity.status(HttpStatus.OK).body(foundedTask);

```

```

        else return ResponseEntity.notFound().build();
    }

    @GetMapping()
    public                               ResponseEntity<List<TaskDto>>
getTasksByCurrentUser(Authentication authentication){
    List<TaskDto> tasks = taskService.getTasksUser(authentication);
    return ResponseEntity.ok(tasks);
}

    @GetMapping("/all")
    public ResponseEntity<List<TaskDto>> getAllTasks(){
    List<TaskDto> tasks = taskService.getAllTasks();
    return ResponseEntity.ok(tasks);
}

    @PostMapping
    public ResponseEntity<?> createTask(@RequestBody TaskDto taskDto,
Authentication auth){
    try{
        TaskDto createdTask = taskService.createTask(taskDto, auth);
        return
ResponseEntity.status(HttpStatus.CREATED).body(createdTask);
    }
    catch (UserPrincipalNotFoundException e){
        return
ResponseEntity.status(HttpStatus.CONFLICT).body("ERROR:    USER    NOT
FOUND");
    }
}

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteTask(@PathVariable(name = "id")
Long id){
    try{
        taskService.deleteTask(id);
    }
    catch (TaskNotFoundException e){
        return ResponseEntity.status(HttpStatus.CONFLICT).build();
    }
}

```

```

        return ResponseEntity.noContent().build();
    }

    @PutMapping("/{id}")
    public ResponseEntity<Void> refreshTask(@PathVariable(name = "id")
Long id, @RequestBody TaskDto taskDto){
        taskDto.setId(id);
        try {
            taskService.updateTask(taskDto);
        } catch (TaskNotFoundException e) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(null);
    }

    @GetMapping("/chains/{username}")
    public ResponseEntity<?> getChains(@PathVariable(name = "username")
String username){
        return
ResponseEntity.ok().body(taskService.getChainsByUser(username));
    }

    @GetMapping("/chains/all")
    public ResponseEntity<?> getChains(){
        return ResponseEntity.ok().body(taskService.getAllChains());
    }

    @PostMapping("/chain/create")
    public ResponseEntity<?> createChain(@RequestBody TasksChainDto
tasksChainDto){
        taskService.createChain(tasksChainDto);
        return ResponseEntity.ok().build();
    }

    @GetMapping("/chain/{id}")
    public ResponseEntity<?> getChain(@PathVariable(name = "id") Long
id){
        return ResponseEntity.ok().body(taskService.getChainById(id));
    }

```

```

        @DeleteMapping("chain/delete/{id}")
        public ResponseEntity<?> deleteChain(@PathVariable Long id){
            taskService.deleteChain(id);
            return ResponseEntity.noContent().build();
        }
        @PostMapping("/chain/edit/{id}")
        public ResponseEntity<?> updateChain(@PathVariable Long id,
        @RequestBody TasksChainDto tasksChainDto){
            taskService.updateChain(id, tasksChainDto);
            return ResponseEntity.ok().build();
        }
    }

    @AllArgsConstructor
    @RestController
    @RequestMapping("/users")
    public class UserController {

        private final UserService userService;

        @PostMapping("/register")
        public ResponseEntity<?> registerUser(@RequestBody UserDto
        userDto){
            try{
                userService.registerUser(userDto);
            }
            catch (UserAlreadyRegisterException e){
                return
                ResponseEntity.status(HttpStatus.CONFLICT).body("Пользователь с таким
                логином уже существует. Попробуйте еще раз.");
            }
            return ResponseEntity.status(HttpStatus.CREATED).build();
        }

        @GetMapping("/all")
        public ResponseEntity<List<UserDto>> showAllUsers(){
            List<UserDto> users = userService.getAllUsers();
            return ResponseEntity.ok(users);
        }
    }

```

```

    }

    @DeleteMapping("/delete/{username}")
    public ResponseEntity<?> deleteUser(@PathVariable(name =
"username") String username){
        try {
            userService.deleteUser(username);
        } catch (UsernameNotFoundException e) {
            return
ResponseEntity.status(HttpStatus.CONFLICT).body("ERROR: USER NOT
FOUND");
        }
        return ResponseEntity.noContent().build();
    }
}

```

```

@RestController
@AllArgsConstructor
@RequestMapping("/auth")
public class AuthController {

    @GetMapping("/me")
    public ResponseEntity<Map<String, Object>>
getAuthorizedUser(Authentication auth) {
        Map<String, Object> user = new LinkedHashMap<>();
        user.put("username", auth.getName());
        user.put("roles",
auth.getAuthorities().stream().map(GrantedAuthority::getAuthority).filter(a
a.contains("ROLE_")).toList());
        user.put("userId", ((User) auth.getPrincipal()).getId());

        return ResponseEntity.ok(user);
    }

    @GetMapping("/success")
    public String success() {
        return "Login successful";
    }
}

```

```

    @GetMapping("/error")
    public String error() {
        return "Login failed";
    }
}

@RestController
@AllArgsConstructor
@RequestMapping("/admin")
public class AdminController {

    private final UserService userService;

    @PutMapping("/reset")
    public ResponseEntity<Void> resetPasswordUser(@RequestBody
UserDto user){
        try{
            userService.updatePassword(user);
        } catch (UsernameNotFoundException e) {
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }
        return ResponseEntity.ok().build();
    }

    @PostMapping("/assign")
    public ResponseEntity<Void> assignRole(@RequestBody UserRoleDto
userRoleDto){
        try{
            userService.assignRole(userRoleDto);
        } catch (Exception e) {
            return ResponseEntity.status(HttpStatus.CONFLICT).build();
        }
        return ResponseEntity.ok().build();
    }

    @PostMapping("/assign/delete")
    public ResponseEntity<Void> withdrawRole(@RequestBody
UserRoleDto userRoleDto){
        try{

```

```

        userService.withdrawRole(userRoleDto);
    } catch (Exception e) {
        return ResponseEntity.status(HttpStatus.CONFLICT).build();
    }
    return ResponseEntity.ok().build();
}
}

```

Репозитории для связи с БД:

```

public interface RoleRepository extends JpaRepository<Role, Long> {
    Optional<Role> findByName(String name);
}

public interface TaskRepository extends JpaRepository<Task, Long> {
    @Query(nativeQuery = true, value = "select * from task t join users u on
t.created_by = u.user_id where u.username = :username")
    List<Task> getTasksOfCurrentUser(String username);
}

public interface TaskStatusRepository extends JpaRepository<TaskStatus,
Long> {
    TaskStatus findByName(String statusName);
}

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findById(Long id);
    boolean existsByUsername(String username);
    Optional<User> findByUsername(String username);
}

```

Сервисы для работы с БД:

```

@Service
@AllArgsConstructor
public class TaskService {

    private final TaskRepository taskRepository;
    private final UserRepository userRepository;
    private final TaskStatusRepository taskStatusRepository;
    private final TaskChainRepository taskChainRepository;

```

```

    private TaskDto taskToDto(Task task){
        TaskDto taskDto = new TaskDto();

```

```

        taskDto.setId(task.getId());
        taskDto.setTitle(task.getTitle());
        taskDto.setDescription(task.getDescription());
        taskDto.setCreated_time(task.getCreatedTime());
        taskDto.setModified_time(task.getChangedTime());
        taskDto.setCreated_by(task.getUser().getUsername());
        taskDto.setStatus(task.getStatus().getStatusName());
        if(task.getChain() != null) {
            taskDto.setChain_id(task.getChain().getId());
            taskDto.setChain_order(task.getChainOrder());
        }
        return taskDto;
    }

```

```

private TasksChainDto toChainDto(TasksChain tasksChain) {
    TasksChainDto tasksChainDto = new TasksChainDto();
    tasksChainDto.setId(tasksChain.getId());
    tasksChainDto.setTitleChain(tasksChain.getTitleChain());
    tasksChainDto.setDeadlineTime(tasksChain.getDeadlineTime());

```

```

    tasksChainDto.setTasksChain(tasksChain.getTasksChain().stream().map(this::task
    ToDto).toList());
    return tasksChainDto;
}

```

```

public Optional<TaskDto> getTaskById(long id){
    Optional<Task> task = taskRepository.findById(id);
    return task.map(this::taskToDto);
}

```

```

    public TaskDto createTask(TaskDto taskDto, Authentication
    authentication) throws UserPrincipalNotFoundException{
        Optional<User> user =
        userRepository.findByUsername(authentication.getName());
        if(user.isEmpty()) throw new
        UserPrincipalNotFoundException(authentication.getName());
        User foundedUser = user.get();
        Task task = new Task();

```

```

        task.setTitle(taskDto.getTitle());
        task.setCreatedTime(LocalDateTime.now());
        task.setChangedTime(LocalDateTime.now());
        task.setDescription(taskDto.getDescription());

task.setStatus(taskStatusRepository.findByStatusName(taskDto.getStatus()));
        task.setUser(foundedUser);
        taskRepository.save(task);
        return taskToDto(task);
    }

    public List<TaskDto> getTasksUser(Authentication authentication){
        List<Task> tasks = taskRepository.findByUser(authentication.getName());
        return tasks.stream().map(this::taskToDto).toList();
    }

    public List<TaskDto> getAllTasks(){
        List<Task> tasks = taskRepository.findAll();
        return tasks.stream().map(this::taskToDto).toList();
    }

    public void deleteTask(long id) {
        Optional<Task> foundTask = taskRepository.findById(id);
        if(foundTask.isEmpty()) throw new TaskNotFoundException();
        taskRepository.deleteById(id);
    }

    public void updateTask(TaskDto taskDto) throws TaskNotFoundException
    {
        Optional<Task> foundTask = taskRepository.findById(taskDto.getId());
        if(foundTask.isEmpty()) throw new TaskNotFoundException();
        Task task = foundTask.get();
        task.setTitle(taskDto.getTitle());
        task.setChangedTime(LocalDateTime.now());
        task.setDescription(taskDto.getDescription());

task.setStatus(taskStatusRepository.findByStatusName(taskDto.getStatus()));
        taskRepository.save(task);
    }

```

```

    }

    public List<TasksChainDto> getChainsByUser(String username){
        List<TasksChain> res =
taskChainRepository.findChainsByUserUsername(username);
        return res.stream().map(this::toChainDto).toList();
    }

    public List<TasksChainDto> getAllChains(){
        List<TasksChain> res = taskChainRepository.findAll();
        return res.stream().map(this::toChainDto).toList();
    }

    public void createChain(TasksChainDto tasksChainDto){
        TasksChain newChain = new TasksChain();
        newChain.setTitleChain(tasksChainDto.getTitleChain());
        newChain.setDeadlineTime(tasksChainDto.getDeadlineTime());
        int order = 1;
        for(TaskDto taskDto : tasksChainDto.getTasksChain()){
            Task task = new Task();
            task.setTitle(taskDto.getTitle());
            task.setDescription(taskDto.getDescription());

            task.setChain(newChain);
            task.setChainOrder(order++);

            task.setCreatedTime(LocalDateTime.now());
            task.setChangedTime(LocalDateTime.now());
            task.setStatus(taskStatusRepository.findByStatusName("OPEN"));

task.setUser(userRepository.findByUsername(taskDto.getCreated_by()).get());
            task.setChainOrder(task.getChainOrder());

            newChain.addTask(task);
        }
        taskChainRepository.save(newChain);
    }

    public void deleteChain(long id){
        taskChainRepository.deleteById(id);

```

```

    }
    public TasksChainDto getChainById(long id){
        Optional<TasksChain> chain = taskChainRepository.findById(id);
        return chain.map(this::toChainDto).get();
    }

    public void updateChain(Long chainId, TasksChainDto dto) {
        TasksChain chain = taskChainRepository.findById(chainId)
            .orElseThrow(() -> new RuntimeException("Цепочка не
найден"));

        chain.setTitleChain(dto.getTitleChain());
        chain.setDeadlineTime(dto.getDeadlineTime());

        Map<Long, Task> existingTasksMap = chain.getTasksChain().stream()
            .filter(task -> task.getId() > 0)
            .collect(Collectors.toMap(Task::getId, Function.identity()));

        Set<Long> incomingTaskIds = new HashSet<>();
        int order = 0;

        for (TaskDto taskDto : dto.getTasksChain()) {

            if (taskDto.getId() != null &&
existingTasksMap.containsKey(taskDto.getId())) {

                Task existingTask = existingTasksMap.get(taskDto.getId());

                existingTask.setTitle(taskDto.getTitle());
                existingTask.setDescription(taskDto.getDescription());
                existingTask.setChangedTime(LocalDateTime.now());
                existingTask.setChainOrder(order);

                if (taskDto.getCreated_by() != null) {
                    User user =
userRepository.findByUsername(taskDto.getCreated_by())
                        .orElseThrow(() -> new RuntimeException("Пользователь
не найден"));
                    existingTask.setUser(user);

```

```

        }
        incomingTaskIds.add(taskDto.getId());

    } else {
        Task newTask = new Task();
        newTask.setTitle(taskDto.getTitle());
        newTask.setDescription(taskDto.getDescription());
        newTask.setCreatedTime(LocalDateTime.now());
        newTask.setChangedTime(LocalDateTime.now());
        newTask.setChainOrder(order);

        if (taskDto.getCreated_by() != null) {
            User user =
userRepository.findByUsername(taskDto.getCreated_by())
                .orElseThrow(() -> new RuntimeException("Пользователь
не найден"));
            newTask.setUser(user);
        }

        chain.addTask(newTask);
    }
    order++;
}
chain.getTasksChain().removeIf(task
!incomingTaskIds.contains(task.getId()));
->

    taskChainRepository.save(chain);
}
}
@AllArgsConstructor
@Service
public class UserService implements UserDetailsService {

    private final UserRepository userRepository;
    private final RoleRepository roleRepository;
    private final BCryptPasswordEncoder bCryptPasswordEncoder;

    private RoleDto toRoleDto(Role role){
        RoleDto roleDto = new RoleDto();

```

```

        roleDto.setId(role.getId());
        roleDto.setName(role.getName());
        return roleDto;
    }

    private UserDto toUserDto(User user) {
        UserDto userDto = new UserDto();
        userDto.setId(user.getId());
        userDto.setUsername(user.getUsername());
        userDto.setPassword(user.getPassword());
        userDto.setRegisterDate(user.getRegisterDate());
        userDto.setBirthDate(user.getBirthDate());
        userDto.setFirstName(user.getFirstName());
        userDto.setLastName(user.getLastName());

        userDto.setRoles(user.getRoles().stream().map(this::toRoleDto).toList());
        return userDto;
    }

    public void registerUser(UserDto userDto) throws
    UserAlreadyRegisterException{
        if(userRepository.existsByUsername(userDto.getUsername())) throw
        new UserAlreadyRegisterException();
        User user = new User();
        user.setUsername(userDto.getUsername());

        user.setPassword(bCryptPasswordEncoder.encode(userDto.getPassword()));
        user.setRegisterDate(LocalDate.now());
        user.setBirthDate(userDto.getBirthDate());
        user.setFirstName(userDto.getFirstName());
        user.setLastName(userDto.getLastName());

        List<Role> roles = new ArrayList<>();
        if(userDto.getUsername().equalsIgnoreCase("admin"))
        roles.add(roleRepository.findByName("ROLE_ADMIN").get());
        else roles.add(roleRepository.findByName("ROLE_USER").get());
        user.setRoles(roles);

        user.setTasks(new ArrayList<>());
    }

```

```

        userRepository.save(user);
    }

    public List<UserDto> getAllUsers(){
        List<User> users = userRepository.findAll();
        return users.stream().map(this::toUserDto).toList();
    }

    public void updatePassword(UserDto userDto) throws
    UsernameNotFoundException{
        Optional<User> foundedUser =
        userRepository.findByUsername(userDto.getUsername());
        if(foundedUser.isEmpty()) throw new
        UsernameNotFoundException(userDto.getUsername());
        User user = foundedUser.get();

        user.setPassword(bCryptPasswordEncoder.encode(userDto.getPassword()));
        userRepository.save(user);
    }

    public void deleteUser(String username) throws
    UsernameNotFoundException{
        if(username.equalsIgnoreCase("admin")) throw new
        UsernameNotFoundException(username);
        userRepository.delete(loadUserByUsername(username));
    }

    public void assignRole(UserRoleDto userDto) throws
    UsernameNotFoundException, RoleNotFoundException{
        Optional<User> foundedUser =
        userRepository.findByUsername(userDto.getUsername());
        if(foundedUser.isEmpty()) throw new
        UsernameNotFoundException(userDto.getUsername());
        Optional<Role> foundedRole =
        roleRepository.findByName(userDto.getRole());
        if(foundedRole.isEmpty()) throw new
        RoleNotFoundException(userDto.getRole());
        User user = foundedUser.get();

```

```

        List<Role> roles = user.getRoles();
        if(!roles.contains(foundedRole.get())) {
            roles.add(foundedRole.get());
            user.setRoles(roles);
            userRepository.save(user);
        }
    }
}

```

```

    public void withdrawRole(UserRoleDto userDto) throws
    UsernameNotFoundException, RoleNotFoundException{
        Optional<User> foundedUser =
        userRepository.findByUsername(userDto.getUsername());
        if(foundedUser.isEmpty()) throw new
        UsernameNotFoundException(userDto.getUsername());
        Optional<Role> foundedRole =
        roleRepository.findByName(userDto.getRole());
        if(foundedRole.isEmpty()) throw new
        RoleNotFoundException(userDto.getRole());
        User user = foundedUser.get();
        List<Role> roles = user.getRoles();
        roles.remove(foundedRole.get());
        user.setRoles(roles);
        userRepository.save(user);
    }
}

```

```

@Override
    public User loadUserByUsername(String username) throws
    UsernameNotFoundException {
        Optional<User> user = userRepository.findByUsername(username);
        if (user.isEmpty()) throw new
        UsernameNotFoundException(username);
        return user.get();
    }
}

```

Основные компоненты клиентской части системы:

tasks-component.html

<div class="tasks-wrapper">

```

<h2 class="page-title">
  Доступные задачи
</h2>

@if (allChains.length > 0) {
<section class="section">
  <h3 class="section-title">
    Цепочки задач
    <span class="badge-count">{{ allChains.length }}</span>
  </h3>

  <div class="accordion custom-accordion" id="chainsAccordion">
    @for (chain of allChains; track chain.id) {
      <div class="accordion-item">
        <h2 class="accordion-header" [id]="heading-chain-' + chain.id">
          <button
                                class="accordion-button"
[class.collapsed]="!isChainExpanded(chain.id)"
      [class.is-chain-completed]="isChainCompleted(chain)"
type="button" (click)="toggleChain(chain.id)"
      [attr.aria-expanded]="isChainExpanded(chain.id)"      [attr.aria-
controls]="collapse-chain-' + chain.id">

          <span class="d-flex align-items-center gap-2 flex-grow-1">
            @if (isChainCompleted(chain)) {
              <i class="bi bi-check-circle-fill text-success fs-5"></i>
            }
            <span class="chain-title-text">{{ chain.titleChain }}</span>
            <span class="badge-count light">
              {{ chain.tasksChain.length }} {{ chain.tasksChain.length | plural:
['задача', 'задачи', 'задач'] }}
            </span>
          </span>

          @if (authService.hasRoles(['ROLE_ADMIN',
'ROLE_MODERATOR'])) {
            <div class="chain-actions">
              <button
                                class="btn-action
                                btn-danger"
(click)="deleteChain(chain.id!)" title="Удалить цепочку">
                <i class="bi bi-trash"></i>
              </button>
            </div>
          }
        </div>
      </div>
    }
  </div>
}

```

```

        </button>
        <button class="btn-action btn-primary"
(click)="editChain(chain.id!)" title="Изменить цепочку">
            <i class="bi bi-pencil"></i>
        </button>
    </div>
}

@if (chain.deadlineTime) {
    <span class="chain-deadline">
        <i class="bi bi-clock"></i> {{ chain.deadlineTime |
date:'dd.MM.yyyy' }}
    </span>
}
</button>
</h2>

<div [id]="collapse-chain-' + chain.id" class="accordion-collapse
collapse"
[class.show]="isChainExpanded(chain.id)" [attr.aria-
labelledby]="heading-chain-' + chain.id"
data-bs-parent="#chainsAccordion">

    <div class="accordion-body">
        <ol class="list-group list-group-numbered custom-list">
            @for (task of chain.tasksChain; track task.id) {
                <li class="list-group-item d-flex justify-content-between align-
items-center"
[class.task-completed]="task.status === 'DONE'" [class.task-
incompleted]="task.status !== 'DONE'">

                    <span class="flex-grow-1 cursor-pointer"
(click)="openTaskDetails(task, chain)"
[title]="IIIar ' + (task.chain_order! + 1)">
                        {{ task.title }}
                    </span>
                </li>
            }
        </ol>

```

```

    </div>
  </div>
</div>
}
</div>
</section>
}

```

```

@if (tasks.length > 0) {
<div class="controls-panel">
  <div class="control-group">
    <label for="sortField">Сортировать по:</label>
    <div class="select-wrapper">
      <select id="sortField" [(ngModel)]="sortField" class="form-select">
        <option value="created_time">Дате создания</option>
        <option value="modified_time">Дате изменения</option>
        <option value="title">Имени (названию)</option>
      </select>
      <span class="select-arrow">▼</span>
    </div>

    <div class="select-wrapper">
      <select [(ngModel)]="sortOrder" class="form-select">
        <option value="desc">По убыванию</option>
        <option value="asc">По возрастанию</option>
      </select>
      <span class="select-arrow">▼</span>
    </div>
  </div>

  <div class="control-group">
    <label for="groupBy">Группировать по:</label>
    <div class="select-wrapper">
      <select id="groupBy" [(ngModel)]="groupBy" class="form-select">
        <option value="none">Без группировки</option>
        <option value="status">Статусу</option>
        @if(authService.hasRoles(['ROLE_ADMIN',
'ROLE_MODERATOR'])) {
          <option value="user">Пользователю</option>

```

```

    }
  </select>
  <span class="select-arrow">▼</span>
</div>
</div>
</div>
}

```

```

<section class="section">
  <h3 class="section-title">
    Задачи вне цепочек
    <span class="badge-count">{{ standaloneTasks.length }}</span>
  </h3>

```

```

  @if (standaloneTasks.length === 0 && allChains.length === 0) {
    <div class="empty-state">
      <p class="empty-text">Задач пока нет</p>
      <p class="empty-hint">Создайте первую задачу, чтобы начать
работу!</p>
      <button class="btn-edit" (click)="createTask()">
        <i class="bi bi-plus-lg"></i> Создать задачу
      </button>
    </div>
  } @else if (standaloneTasks.length === 0) {
    <div class="info-hint">
      <i class="bi bi-info-circle me-2"></i> Все задачи распределены по
цепочкам
    </div>
  } @else {
    <div class="tasks-grid">
      @for (group of processedTasks; track group.key) {
        @if (groupBy !== 'none') {
          <h4 class="group-title">
            @switch (group.key) {
              @case ('OPEN') { Открыто }
              @case ('CLOSED') { Закрыто }
              @case ('IN_PROGRESS') { В процессе }
              @case ('DONE') { Завершено }
              @default { {{ group.key }} }

```

```

    }
    <span class="group-count">({{ group.tasks.length }})</span>
</h4>
}

<div class="group-tasks">
    @for (task of group.tasks; track task.id) {
        <app-task-card-component [task]="task"
(deleteTask)="deleteTask($event)" (changeTask)="updateTask($event)">
        </app-task-card-component>
    }
</div>
}
</div>
}
</section>

@if (selectedTask) {
    <app-info-task-modal [task]="selectedTask" [chain]="selectedTaskChain"
(confirm)="confirmTask($event)"
(close)="closeModal()" (change)="updateTask($event)"
(delete)="deleteTask($event)">
    </app-info-task-modal>
}
</div>

```

```

tasks-component.ts
@Component({
    selector: 'app-tasks-component',
    standalone: true,
    imports: [CommonModule, TaskCardComponent, FormsModule,
InfoTaskModal, PluralPipe],
    templateUrl: './tasks-component.html',
    styleUrls: ['./tasks-component.css'],
})
export class TasksComponent implements OnInit {
    tasks: Task[] = [];
    allChains: ChainTasks[] = [];
    expandedChains = new Set<number>();
}

```

```
sortField: 'title' | 'created_time' | 'modified_time' = 'created_time';
sortOrder: 'asc' | 'desc' = 'desc';
groupBy: 'none' | 'status' | 'user' = 'none';
```

```
authService = inject(AuthService);
private taskService = inject(TaskService);
private router = inject(Router);
private cd = inject(ChangeDetectorRef);
```

```
selectedTask: Task | null = null;
selectedTaskChain: ChainTasks | null = null;
```

```
ngOnInit(): void {
  this.loadTasks();
}
```

```
private sortTasks(tasks: Task[]): Task[] {
  return [...tasks].sort((a, b) => {
    if (this.sortField === 'title') {
      const valA = (a.title || '').toLowerCase();
      const valB = (b.title || '').toLowerCase();
      return this.sortOrder === 'asc'
        ? valA.localeCompare(valB)
        : valB.localeCompare(valA);
    } else {
      const dateA = a[this.sortField];
      const dateB = b[this.sortField];
      const valA = dateA ? new Date(dateA).getTime() : 0;
      const valB = dateB ? new Date(dateB).getTime() : 0;
      return this.sortOrder === 'asc' ? valA - valB : valB - valA;
    }
  });
}
```

```
get processedTasks(): { key: string; tasks: Task[] }[] {
  if (!this.tasks || this.tasks.length === 0) return [];
```

```
  let tasks = [...this.standaloneTasks];
```

```

const sorted = this.sortTasks(tasks);

if (this.groupBy === 'none') {
  return [{ key: 'Все задачи', tasks: sorted }];
}

const groups = new Map<string, Task[]>();

sorted.forEach(task => {
  let groupKey = 'Без группы';

  if (this.groupBy === 'status') {
    groupKey = task.status;
  } else if (this.groupBy === 'user') {
    groupKey = task.created_by || 'Неизвестный пользователь';
  }

  if (!groups.has(groupKey)) {
    groups.set(groupKey, []);
  }
  groups.get(groupKey)!.push(task);
});
return Array.from(groups, ([key, tasks]) => ({ key, tasks }));
}

get standaloneTasks(): Task[] {
  return this.sortTasks(this.tasks.filter(t => !t.chain_id));
}

get taskChains(): ChainTasks[] {
  const chainMap = new Map<number, ChainTasks>();

  for (const chain of this.allChains) {
    chainMap.set(chain.id!, {
      id: chain.id,
      titleChain: chain.titleChain,
      tasksChain: chain.tasksChain,
      deadlineTime: chain.deadlineTime
    });
  }
}

```

```

    });
}

const chains = Array.from(chainMap.values());
return chains;
}

openTaskDetails(task: Task, chain?: ChainTasks): void {
    this.selectedTask = task;
    this.selectedTaskChain = chain ?? null;
    document.body.style.overflow = 'hidden';
}

closeModal(): void {
    this.selectedTask = null;
    this.selectedTaskChain = null;
    document.body.style.overflow = "";
}

toggleChain(chainId?: number): void {
    if (this.expandedChains.has(chainId!)) {
        this.expandedChains.delete(chainId!);
    } else {
        this.expandedChains.add(chainId!);
    }
}

isChainExpanded(chainId?: number): boolean {
    return this.expandedChains.has(chainId!);
}

private loadTasks(): void {
    const isAdminOrMod = this.authService.hasRoles(['ROLE_ADMIN',
'ROLE_MODERATOR']);
    const request$ = isAdminOrMod
        ? this.taskService.getAllTasks()
        : this.taskService.getTasks();

    request$.subscribe({

```

```

    next: (data) => { this.tasks = data },
    error: (err) => console.error('Ошибка загрузки задач:', err),
    complete: () => this.cd.markForCheck()
  });

  const request2$ = isAdminOrMod
    ? this.taskService.getAllChains()
    :
  this.taskService.getChains(this.authService.getCurrentUser()?.username!);

  request2$.subscribe({
    next: (data) => { this.allChains = data },
    complete: () => this.cd.markForCheck()
  });
}

deleteTask(id: number): void {
  if (!confirm('Удалить задачу?')) return;

  this.taskService.deleteTask(id).subscribe({
    next: () => {
      this.tasks = this.tasks.filter(task => task.id !== id);
      this.cd.markForCheck();
    },
    error: (err) => console.error('Ошибка удаления задачи:', err)
  });
}

updateTask(id: number): void {
  this.router.navigate(['/tasks', id]);
}

confirmTask(task: Task): void {
  task.status = 'DONE';
  this.taskService.updateTask(task).subscribe({
    next: (response) => {
      this.router.navigate(['/tasks']);
    },
    error: (error) => {

```

```

        alert('Ошибка изменения задачи.')
        console.error('Error updating task:', error);
    }
});
}

```

```

deleteChain(id: number): void {
    this.taskService.deleteChain(id).subscribe({
        next: () => {
            this.cd.markForCheck();
        },
        error: (err) => console.error('Ошибка удаления цепочки:', err),
        complete: () => this.cd.markForCheck()
    });
}

```

```

createTask(): void {
    this.router.navigate(['/tasks/new']);
}

```

```

editChain(id: number): void {
    this.router.navigate(['/chain/', id]);
}

```

```

isChainCompleted(chain: any): boolean {
    if (!chain.tasksChain || chain.tasksChain.length === 0) {
        return false;
    }
    return chain.tasksChain.every((task: any) => task.status === 'DONE');
}

```

task-service.ts

```

@Injectable({
    providedIn: 'root',
})

```

```

export class TaskService {
    private http = inject(HttpClient);
    private url: string = environment.apiUrl;
    private router = inject(Router);

```

```

    private taskChangedSubject = new Subject<void>();

```

```
taskChanged$ = this.taskChangedSubject.asObservable();
```

```
getTasks(): Observable<Task[]> {  
  return this.http.get<Task[]>(this.url + "/tasks");  
}
```

```
getAllTasks(): Observable<Task[]> {  
  return this.http.get<Task[]>(this.url + "/tasks/all");  
}
```

```
getTask(id: number): Observable<Task> {  
  return this.http.get<Task>(this.url + "/tasks/" + id);  
}
```

```
deleteTask(id: number): Observable<any> {  
  return this.http.delete(this.url + "/tasks/" + id).pipe(  
    tap(() => this.taskChangedSubject.next())  
  );  
}
```

```
createTask(task: Task): Observable<Task> {  
  return this.http.post<Task>(this.url + "/tasks", task).pipe(  
    tap(() => this.taskChangedSubject.next())  
  );  
}
```

```
updateTask(task: Task): Observable<any> {  
  return this.http.put<any>(this.url + "/tasks/" + task.id, task).pipe(  
    tap(() => this.taskChangedSubject.next())  
  );  
}  
}
```

```
sidebar-wrapper.component.ts
```

```
declare global {  
  interface Window {  
    sidebar_app?: any;  
  }  
}
```

```

@Component({
  selector: 'app-sidebar-wrapper',
  template: `<div id="sidebar-root"></div>`,
  styles: [
    :host {
      display: block;
      height: 100%;
    }
  ]
})
export class SidebarWrapperComponent implements OnInit, OnDestroy,
OnChanges {
  @Input() user: User | null = null;
  @Input() tasks: Task[] = [];

  private reactRoot: any = null;
  private SidebarComponent: any = null;
  sidebarTasks: Task[] = [];
  authUser: User | any = null;

  private authService = inject(AuthService);
  private taskService = inject(TaskService);
  private router = inject(Router);

  private taskUpdateSubscription!: Subscription;
  private userSubscription!: Subscription;

  async ngOnInit() {
    await this.loadSidebar();

    this.taskUpdateSubscription = this.taskService.taskChanged$.subscribe(()
=> {
      this.loadSidebarTasks();
    });

    this.userSubscription = this.authService.currentUser$.subscribe(user => {
      this.authUser = user;
      if (user) {

```

```

        this.loadSidebarTasks();
    }
});
}

onLogoutHandle(): void {
    this.authService.logout();
}

onNavigateHandle(path: string | undefined): void {
    this.router.navigate([path]);
}

private loadSidebarTasks(): void {
    const isAdminOrMod = this.authService.hasRoles(['ROLE_ADMIN',
'ROLE_MODERATOR']);
    const isUser = this.authService.hasRoles(['ROLE_USER']);

    if (!isAdminOrMod && !isUser) return; // Пользователь ещё не
аутентифицирован

    const request$ = isAdminOrMod
        ? this.taskService.getAllTasks()
        : this.taskService.getTasks();

    request$.subscribe({
        next: (tasks) => {
            this.tasks = tasks.slice(0, 5).reverse();
            this.renderSidebar();
        },
        error: (err) => {
            console.error('Ошибка загрузки задач для сайдбара', err);
            this.sidebarTasks = [];
            this.renderSidebar();
        }
    });
}

ngOnDestroy() {

```

```

this.taskUpdateSubscription?.unsubscribe();
this.userSubscription?.unsubscribe();

if (this.reactRoot && this.reactRoot.unmount) {
  this.reactRoot.unmount();
}
}

private async loadSidebar() {
  try {
    const script = document.createElement('script');
    script.src = 'http://localhost:3001/remoteEntry.js';
    script.type = 'text/javascript';

    await new Promise((resolve, reject) => {
      script.onload = resolve;
      script.onerror = reject;
      document.head.appendChild(script);
    });

    if (window.sidebar_app) {
      await window.sidebar_app.init({
        shareScope: 'default',
      });
    }

    const factory = await window.sidebar_app.get('./Sidebar');
    const module = factory();

    this.SidebarComponent = module.default || module;
  } catch (error) {
    console.error('Failed to load sidebar microfrontend:', error);
  }
}

private renderSidebar() {
  if (!this.SidebarComponent) return;

  const container = document.getElementById('sidebar-root');

```

```

    if (!container) return;

    const props = {
      user: {
        username: this.authUser?.username,
        roles:      this.authUser?.roles?.toString()?.replace('ROLE_USER',
'Пользователь').replace('ROLE_ADMIN',
'Администратор').replace('ROLE_MODERATOR', 'Модератор').replace(',', ' '),
      },
      tasks: this.tasks.map(t => ({
        id: t.id,
        title: t.title,
        status: t.status
      })),
      onLogout: () => this.onLogoutHandle(),
      onNavigate: (path: string) => this.onNavigateHandle(path)
    };

    const React = (window as any).React;
    const ReactDOM = (window as any).ReactDOM;

    const element = React.createElement(this.SidebarComponent, props);

    if (!this.reactRoot) {
      this.reactRoot = ReactDOM.createRoot(container);
    }
    this.reactRoot.render(element);
  }

  ngOnChanges() {
    if (this.SidebarComponent) {
      this.renderSidebar();
    }
  }
}

index.tsx
const root = ReactDOM.createRoot(document.getElementById('root') as
HTMLElement);

```

```

root.render(
  <React.StrictMode>
    <Sidebar
      tasks={}
      user={{username: 'Test'}}
      onLogout={() => console.log('LOGOUT')}
      onNavigate={(path: string) => console.log('Navigate to: ', path)}
    ></Sidebar>
  </React.StrictMode>
);

```

Sidebar.tsx

```

import React from 'react';
import { SidebarProps } from '../types';
import './Sidebar.css';

```

```

const Sidebar: React.FC<SidebarProps> = ({
  tasks,
  user,
  onLogout,
  onNavigate
}) => {

  return (
    <div>
      <aside className="blue-sidebar">
        <div className="sidebar-header">
          <a href="/" className="sidebar-brand">
            <div className="brand-text">Менеджер задач</div>
            <div className="user-roles">Микрофронтенд на React</div>

          </a>
        </div>

        <div className="sidebar-content">

          <nav className="sidebar-nav">
            <ul className="nav-list">
              <li className="nav-item">

```

```

        <a className="nav-link" onClick={() => onNavigate("/tasks")}>
          Задачи
        </a>
      </li>
      <li className="nav-item">
        <a className="nav-link" onClick={() =>
onNavigate("/tasks/new")}>
          Создать задачу
        </a>
      </li>

```

```

      {user?.roles?.includes("Администратор") && (
        <li className="nav-item">
          <a className="nav-link" onClick={() =>
onNavigate("/chain/create")}>
            Создать цепочку
          </a>
        </li>
      )}
      {user?.roles?.includes("Администратор") && (
        <li className="nav-item">
          <a className="nav-link" onClick={() =>
onNavigate("/users/all")}>
            Все пользователи
          </a>
        </li>
      )}
    </ul>
  </nav>

```

```

<div className="sidebar-section">
  <h3 className="section-title">Последние задачи</h3>

```

```

  {tasks.length > 0 ? (
    <div className="sidebar-tasks">
      <ul className="task-list">
        {tasks.slice(0, 5).map((task) => (

```

```

        <li key={task.id} className="task-item" onClick={() =>
onNavigate("/tasks/" + task.id)} >
            <span className={`status-indicator status-${task.status}`} />
            <span className="task-title">{task.title}</span>
        </li>
    )})
</ul>
</div>
): (
    <p className="task-item empty">Задачи пока не
обнаружены.</p>
    )}
</div>
</div >

<div className="sidebar-footer">
    <div className="user-card">
        <div className="user-name">{user?.username}</div>
        <div className="user-roles">{user?.roles}</div>
    </div>
    <button className="btn-sidebar btn-logout" onClick={onLogout}>
        Выйти
    </button>
</div>
</aside>
</div >
);
};

export default Sidebar;

```